

A domain analysis of resource and requirements monitoring: Towards a comprehensive model of the software monitoring domain

Rick Rabiser^{a,*}, Klaus Schmid^b, Holger Eichelberger^b, Michael Vierhauser^c, Sam Guinea^d, Paul Grünbacher^a

^a Christian Doppler Laboratory MEVSS, Institute for Software Systems Engineering, Johannes Kepler University Linz, Austria

^b Software Systems Engineering, University of Hildesheim, Germany

^c Department of Computer Science and Engineering, University of Notre Dame, IN, USA

^d Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Italy

ARTICLE INFO

Keywords:

Software monitoring
Requirements monitoring
Resource monitoring
Domain model
Reference architecture

ABSTRACT

[Context] Complex and heterogeneous software systems need to be monitored as their full behavior often only emerges at runtime, e.g., when interacting with other systems or the environment. Software monitoring approaches observe and check properties or quality attributes of software systems during operation. Such approaches have been developed in diverse communities for various kinds of systems and purposes. For instance, requirements monitoring aims to check at runtime whether a software system adheres to its requirements, while resource or performance monitoring collects information about the consumption of computing resources by the monitored system. Many venues publish research on software monitoring, often using diverse terminology, and focusing on different monitoring aspects and phases. The lack of a comprehensive overview of existing research often leads to re-inventing the wheel. **[Objective]** We provide a domain model to structure and systematize the field of software monitoring, starting with requirements and resource monitoring. **[Method]** We developed an initial domain model based on (i) our extensive experiences with requirements and resource monitoring, (ii) earlier efforts to develop a comparison framework for monitoring approaches, and (iii) an earlier systematic literature review on requirements monitoring frameworks. We then systematically analyzed 47 existing requirements and resource monitoring approaches to iteratively refine the domain model and to develop a reference architecture for software monitoring approaches. **[Results]** Our domain model covers the key elements of monitoring approaches and allows analyzing their commonalities and differences. Together with the reference architecture, our domain model supports the development of integrated monitoring solutions. We provide details on 47 approaches we analyzed with the model to assess its coverage. We also evaluate the reference architecture by instantiating it for five different monitoring solutions. **[Conclusions]** We conclude that requirements and resource monitoring have more commonalities than differences, which is promising for the future integration of existing monitoring solutions.

1. Introduction and motivation

Today's large-scale and complex software systems—such as cyber-physical systems [1], cloud-based systems [2], or systems of systems [3]—often cannot be fully tested during development. Indeed, their full behavior typically only emerges at runtime, when they interact with other systems, hardware, and the environment. Software monitoring approaches [4,5], i.e., techniques and tools that allow to observe and check properties and quality attributes of systems at runtime, thus need to be employed. Monitoring approaches are becoming increasingly prevalent,

particularly due to ever-increasing requirements on availability and runtime reconfiguration capabilities [6–8].

Software monitoring approaches have been developed in diverse communities and for various kinds of systems and purposes. Examples include requirements monitoring [5,9], resource or performance monitoring [10–12], monitoring of architectural properties [13], complex event processing [14], and runtime verification [8,15]. The desired runtime behavior is often formally expressed using temporal logic [16] or domain-specific (constraint) languages [9,17,18]. The defined constraints are checked based on events and information collected from systems at runtime, e.g., through instrumentation [19].

* Corresponding author.

E-mail addresses: rick.rabiser@jku.at (R. Rabiser), schmid@sse.uni-hildesheim.de (K. Schmid), eichelberger@sse.uni-hildesheim.de (H. Eichelberger), mvierhau@nd.edu (M. Vierhauser), sam.guinea@polimi.it (S. Guinea), paul.gruenbacher@jku.at (P. Grünbacher).

<https://doi.org/10.1016/j.infsof.2019.03.013>

Received 16 January 2018; Received in revised form 27 February 2019; Accepted 25 March 2019

Available online 26 March 2019

0950-5849/© 2019 Elsevier B.V. All rights reserved.

The diversity of existing approaches has motivated several attempts to structure the research field. Most notably, Dwyer et al. [20] discussed patterns for specifying properties in runtime verification; Delgado et al. [21] provided a taxonomy and a catalog of tools for fault monitoring; and Autili et al. [22] tried to align qualitative, real-time, and probabilistic property specification patterns. Furthermore, three of the authors of this paper performed a systematic literature review on requirements monitoring frameworks [5], and four of the authors of this paper developed a comparison framework for software monitoring approaches [4]. However, these existing publications were either intended for people from one particular monitoring community, or focused on particular aspects of monitoring, such as specifying the properties to be monitored.

The overall software monitoring research field remains unstructured: a plethora of approaches has been developed in different communities and published at different venues, focusing on different aspects or phases of the monitoring process. As a result, the wheel is often re-invented, as researchers and practitioners find it challenging to identify and integrate potentially useful approaches. This is further hampered by the lack of a common terminology, i.e., different communities often use different terms, even when speaking about the same things. For instance, terms such as “probing”, “instrumentation”, “interception”, and “data extraction” are often used interchangeably, while they all refer to extracting information from a running system for the purpose of monitoring. Both types of software monitoring approaches, requirements monitoring and resource monitoring, share similarities, but focus on different aspects (requirements vs. resources).

The goal of this paper is thus to *structure and systematize the research on software monitoring systems*. More specifically, we present a domain model for software monitoring based on (i) our extensive experiences in the fields of requirements monitoring [18,23] and resource monitoring [11]; (ii) earlier efforts to develop a comparison framework for monitoring approaches [4]; and (iii) an existing systematic literature review on requirements monitoring frameworks [5]. We systematically analyzed 47 monitoring approaches from the requirements and resource monitoring communities to check whether the domain model covers these approaches. During this process, we iteratively refined the domain model and also developed a reference architecture for software monitoring approaches. To evaluate the reference architecture, we created five instances: four representing approaches with at least one of the authors involved, i.e., REMINDS [18], ECOWARE [23], SPASS-meter [11], and the QualiMaster solution [24], and—supported by experienced developers—one additional instance for the Kieker monitoring approach [10,25].

In contrast to existing work structuring the research field [4,5,20–22], our domain model aims to provide a basis to link different monitoring communities, describe heterogeneous approaches, identify overlaps, and support developing integrated solutions. In this paper we apply our model to both requirements and resource monitoring.

Specifically, we claim the following **contributions**:

Contribution 1. We provide a **domain model** that defines the key elements of monitoring systems. The model can be used as a *starting point for further research* on software monitoring, e.g., to include concepts and terms from other domains, such as runtime verification [8,15] or complex event processing [14]. It can help to *link different monitoring communities and research areas* by highlighting their commonalities and differences and *supports analyzing and categorizing existing approaches*. For instance, it allowed us to categorize 47 existing approaches, i.e., 27 requirements monitoring approaches and 20 resource monitoring approaches. The information we extracted is provided in an *online spreadsheet* [26].

Contribution 2. We developed a **reference architecture** outlining the key components and layers of software monitoring solutions. Specifically, we compared the information extracted from 47 approaches to identify common and variable elements.

By allowing systematic comparisons both the domain model and the reference architecture *support and motivate the development of integrated solutions*, e.g., software monitoring approaches that support both requirements and resource monitoring. Both researchers and practitioners can benefit from our work, i.e., researchers can use the domain model to understand the domain of software monitoring and categorize and motivate their own research. Researchers and practitioners can rely on the reference architecture when developing a monitoring solution.

In the remainder of this paper we first describe the background of this work (cf. Section 2): we define key terms and briefly introduce requirements monitoring, resource monitoring, and other software monitoring approaches. We then explain our research method (cf. Section 3). We describe our domain model (cf. Section 4) and the coverage assessment we conducted for the model (cf. Section 5). We then present and explain our reference architecture (cf. Section 6) and its instantiation for five different monitoring approaches (cf. Section 7). We conclude with a discussion of related work (cf. Section 8) and a summary and an outlook on future work (cf. Section 9).

2. Background

In this section we provide a brief overview of requirements monitoring, resource monitoring, and other software monitoring areas. We start by introducing a list of the *terms* we use throughout the paper to define the key elements and stakeholders of a monitoring approach:

- **Monitoring system:** a solution that observes and (optionally) evaluates a particular system, e.g., that instruments a system to extract monitoring information, such as events and/or data, at runtime and reasons on this information. Synonyms: monitoring framework, monitoring solution.
- **Monitored system:** the system that is being monitored by the monitoring system.
- **Instrumentation:** the modification of a monitored system, either before putting it into operation (*static instrumentation*) or while the system is running (*dynamic instrumentation*), to extract monitoring information.
- **Monitoring information:** monitoring information refers to information collected by a monitoring system, e.g., via polling or instrumentation. Such information may include system properties, e.g., available memory or values of object attributes.
- **Event:** An event is a specific type of monitoring information, which describes something that happened in the system. A monitoring system can passively listen to events and/or actively collect them.
- **Event data:** While some monitoring systems solely rely on the name and type of events, others also take into account the payload of events to which we explicitly refer as “event data” or “data attached to events”.
- **(Event) trace:** a stream of events that is collected from the monitored system for reasoning purposes; the stream may be persisted to support further (post-mortem) analyses.
- **Aggregation:** the process of summarizing monitoring information, that has been extracted from the monitored system, to provide information at the right level of granularity for further tasks.
- **Filtering:** the process of selecting monitoring information that is of interest for further tasks; this activity is typically based on explicitly defined conditions.
- **Checking:** the process of reasoning (at runtime) on monitoring information, e.g., to evaluate constraints on events or event data.
- **Monitoring engineer:** the person setting up the monitoring system.
- **Monitoring analyst:** the person analyzing the collected monitoring information.

2.1. Requirements monitoring

Requirements monitoring approaches continuously evaluate a system at runtime to ensure that it adheres to its defined requirements [9].

More than twenty years ago, Fickas and Feather [27] proposed requirements monitoring to support the maintenance of systems in dynamic environments. Their work relies on establishing a relationship between the requirements of a system, assumptions on the actual state of the system, and “evolutions”, i.e., reactions to violations of these assumptions. This is particularly important because software systems are subject to continuous change and ongoing evolution during their lifetime.

Several approaches [5] have been proposed that commonly rely on formalizing requirements, e.g., as constraints, and subsequently checking them using monitoring information. The conceptual framework proposed by Robinson [9] takes into account several different aspects of requirements monitoring, from capturing low-level runtime information to presenting results of requirements validations to end users. It thus facilitates discussing different approaches to requirements monitoring. The framework is structured along four different layers: the *Event Layer* is responsible for retrieving events and information from various sources and for transporting them to a storage facility. Sources include the software to be monitored and its execution environment (cf. Section 2.2). For example, several approaches [13,28–30] use aspect-oriented programming as a primary mechanism to retrieve monitoring information from systems. Others rely on interception [31–33] or developing probes [9,34,35]. The *Model Layer* is responsible for performing analyses on the information captured at runtime. This includes the interpretation of software behavior, or the behavior of system users, based on a model describing the requirements such as goal models [28], (domain-specific) constraint languages [9], or temporal logic [36]. The *Presentation Layer* provides services for presenting the results of the analyses performed at the model layer to the users of the monitoring framework. They are informed about deviations from the expected behavior, i.e., violations of the requirements, so that they can decide on further maintenance or evolution activities. Finally, the *Application Layer* contains tools and application-specific services for various purposes, e.g., for controlling the monitored system by adapting it to fix the detected violations. For example, tools provide dashboards [9,23], generate and present violation reports [31], or send alarms to users [37].

2.2. Resource monitoring

Resource monitoring aims at collecting information about the consumption of computing resources by the monitored system. Traditionally, computing resources are divided into physical resources and logical resources [11]. *Physical resources* are provided by the hardware and can be characterized in terms of utilized CPU time, allocated memory, used disk space, used network bandwidth, consumed electrical power, etc. *Logical resources* are provided by the operating system or the runtime environment, e.g., a virtual machine. Typically, a logical resource is an abstraction of a physical resource or some refined resource concept. Examples include executed programming language statements per time unit (as opposed to CPU instructions), use of swap space or virtual memory, and fractions of physical or logical resources consumed by processes or threads.

Some information about resource consumption can be directly accessed from the operating system or the runtime environment, e.g., the memory utilization of a Java Virtual Machine (JVM). However, the use of certain other resources, such as the response time of a method or service, can only be determined by observing specific details of the monitored system. Such information is usually provided by a *monitoring probe* [11,38,39]. In some cases, the required information can be accessed through an API or a service provided by the monitored system. In model-based software development, monitoring probes can be added through *generating* [24] additional code. However, the monitored system is often not prepared for monitoring, and probes need to be added through *instrumentation*. Typically, monitoring probes add a certain *overhead* to normal operation, e.g., affecting the computational power [40–42] or the memory consumption [40,43] of the monitored system. To achieve low overhead, resource monitoring approaches typically do not

aim at complex in-vivo *online analyses*, e.g., aggregation of monitoring information or constraint checking. Instead, many tools collect extensive traces, e.g., using Open.XTrace [44], in order to support different post-mortem *offline analyses*.

2.3. Other monitoring approaches

A wide variety of (software) monitoring approaches exist besides resource and requirements monitoring. This includes, for example, *monitoring properties of physical entities* such as network traffic [45–47], *verifying systems at runtime* (e.g., security, safety, and privacy properties [48,49]), or *monitoring cyber-physical systems* [50]. Most of these approaches use formal methods and models. They also overlap with approaches for requirements monitoring (cf. Section 2.1) and resource monitoring (cf. Section 2.2). For example, requirements monitoring approaches may also be used for monitoring security requirements, while resource monitoring approaches for software may also be used for monitoring physical entities. These overlaps make the development of a common domain model very promising. Due to our own background and experience, however, we focus on requirements monitoring and resource monitoring in this paper.

Besides approaches and tools developed in academia, a number of *commercial tools* exist, which support monitoring of different types of systems and resources. In the area of application performance management, for example, Dynatrace¹ provides a solution for full-stack monitoring, which comprises application performance management, and infrastructure monitoring of services, servers and cloud infrastructures. Similarly, tools such as NewRelic², Foglight³ and AppDynamics⁴ also focus on monitoring cloud environments and (business) performance. In this paper we do not provide further details on these approaches and tools, since our focus is explicitly on research approaches for requirements monitoring and resource monitoring as detailed above.

3. Research method

The main goal of this work is to create a comprehensive overview of the monitoring landscape going beyond existing work focusing on specific sub-domains, such as resource monitoring (e.g., for cloud-based systems [51] or grid-based systems [52]) or requirements monitoring [4,5]. As the various kinds of monitoring approaches deal with significantly overlapping problems, sometimes even down to specific technical details, we decided to develop an integrated overview of the different kinds of requirements and resource monitoring systems.

Fig. 1 summarizes the research process we followed. Our first step was to specify the goal and scope of our study. The next step was to create an initial domain model. The main inputs for this step were our experiences and earlier works on conducting a systematic literature review [5] and a comparison framework [4]. We systematically explored the capabilities and characteristics of a generic monitoring approach through several brainstorming sessions and workshops conducted over a period of two weeks.

After developing this initial domain model, we collected literature relevant for our study, i.e., papers describing requirements and resource monitoring approaches. The approaches analyzed in the previously mentioned systematic literature review on requirements monitoring [5] served as one major source. Additionally, we included papers that had already previously been collected for overviews of resource monitoring [11]. We identified additional literature through snowballing, i.e., following references of the already included studies. This led to a grand total of 47 approaches (cf. our online spreadsheet [26]):

¹ <http://www.dynatrace.com>.

² <http://www.newrelic.com>.

³ <https://www.quest.com/foglight/>.

⁴ <http://www.appdynamics.com>.

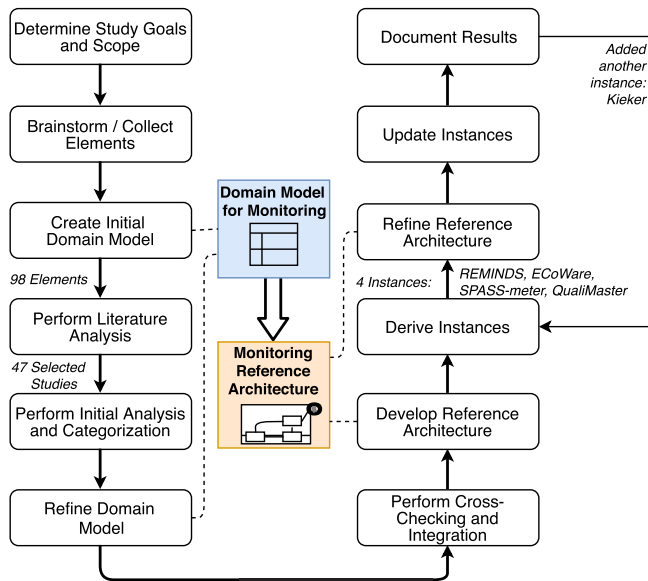


Fig. 1. Research process.

27 requirements monitoring approaches and 20 resource monitoring approaches. Our intention was not to collect all existing approaches, but to analyze a significant number of highly relevant approaches from both domains. We believe that 47 approaches are sufficient for the purpose of our study.

We distributed the 47 approaches among the six authors of this paper, who then classified them according to the domain model. Specifically, each author read the papers in detail and extracted information regarding the elements of our domain model. In case of doubt, discussions among several authors took place (cf. cross-checking phase below). As some authors of this paper had not been deeply involved in creating

the initial model, this also served as a cross-checking step regarding the clarity of the description of the different aspects of the domain model, especially in terms of the used terminology.

Observations in this analysis then led to refinements and refactorings of the model. However, these modifications were surprisingly minor, probably due to the extensive prior experience the authors of this paper had with the topic area. It should be noted that we did not intend the domain model to be a one-on-one representation of the selected studies. Specifically, when developing the domain model, we included elements that could be useful based on our experience (and kept those elements in the model), even if we could not find any specific approach in the 47 studies that already implemented these elements.

We performed a cross-check to ensure that the categorizations are adequate: for every approach two authors of this article independently extracted information on the elements of the domain model. They then compared and discussed what they had extracted to reach mutual agreement. This addressed the dual goal of ensuring (i) that the existing categorizations for approaches were reliable and (ii) that the domain model could be applied in a repeatable manner.

The result of this cross-check was reintegrated into the domain model. At that point we also started creating a reference architecture for monitoring solutions. For this purpose, in a meeting between two of the authors of this paper we compared all the information we had extracted for the 47 approaches using our domain model and drafted a first version of a reference architecture on the whiteboard, step-by-step adding elements frequently occurring in the 47 approaches. The creation of the reference architecture was mainly informed by elements of the domain model related with the collection of monitoring information, aggregation, filtering, and checking (highlighted in italics and blue color in Tables 1–5). We iteratively refined this first draft during multiple meetings.

The process of architecture development also helped identify some minor issues in the domain model and led to corresponding iterations as shown in Fig. 1. The resulting architecture is described in Section 6. In order to further validate this architecture—and, thus indirectly the

Table 1

Domain model of the field of monitoring software systems (1/5): life-cycle support and scoping. Name and Purpose categorize monitoring approaches.

Categorization information	Definition/Example
Name (NAM) (cf. [4].) Purpose(s) (PUR) (cf. [4].) • Undefined • Monitoring • <i>Resource Monitoring</i> • <i>Requirements Monitoring</i> • <i>Other Mon. Approaches</i> • Adaptation • Evolution	name of the monitoring approach or author(s) what is the purpose of the monitoring approach? no purpose defined the approach focuses on monitoring software systems it monitors resources such as memory, CPU, network, etc. and/or focuses on monitoring performance it monitors (and checks) properties and/or behavior of a software system extend here for other types of monitoring approaches the approach focuses on adapting a software system (based on information gathered through monitoring) the approach focuses on supporting evolution of a software system (based on information gathered through monitoring)
Domain Model Element (ID)	Definition/Example
Life-Cycle support (LCS) (cf. [4].) • Requirements engineering • Design • Implementation • Testing (validation & verification during development) • Commissioning • Operations (Ops) • Evolution	to which phases of the development of the monitored system does the use of the monitoring approach contribute monitoring approach contributes to defining/eliciting requirements for the monitored system monitoring approach contributes to defining architecture/design models for the monitored system, e.g., communication behavior of components defined via constraints (later being monitored) debug the current system state using the monitoring approach validation and/or verification of the monitored system using the monitoring approach support for the deployment of monitored software to hardware/environment, e.g., acceptance tests support for the operation of the monitored system, e.g., checking behavioral constraints at runtime support for further developing the monitored system based on background information learned through monitoring
Supported phases (SPH) • Scoping (SCO) • Scope element • Hardware • SW-Component/Service • Platform • Runtime • System • Relation	phases of the monitoring process supported by the approach what to monitor from where? what elements to monitor e.g., server, sensors/actuators, network parts of the system and their communication e.g., application Server/DBMS e.g., JVM, CLR, Android VMs explicitly monitor the overall system a relation between two or more elements that can be directly monitored as separate entity – Example: how much data goes from x to y (not just what x sends and y receives)

Table 2

Domain model (2/5): collection of monitoring information, aggregation, filtering/checking.

Domain model element (ID)	Definition/Example
• <i>Collection of monitoring information (COL)</i> * 3rd party or hidden * Probe insertion · manual · automatic * Probe generation · manual · automatic * When is monitoring information collected · Periodical/Sampling · Continuous · On demand • <i>Aggregation (AGG)</i> • <i>Filtering/Checking (FIC)</i> * <i>When does filtering/checking happen (TFI)</i> · Online · periodical · continuous · occasional/on demand · Offline (trace analysis)	how to collect monitoring information like events or event data? Includes the instrumentation process probing supported by third-party technology (e.g., JMX) or hidden by the monitoring framework (not transparent for the user of the framework) how probes are inserted into the monitored target system (e.g., code changed manually vs. AspectJ, ASM, JavaAssist) how probes are written (e.g., write in programming language vs. declarative definition + generation) information like events/event data collected at defined points in time or based on certain defined conditions information collected as soon as available (e.g., events/event data collected as they occur) possibility to actively ask for monitoring information (e.g., <code>getEvents()</code>, <code>getInformationCollected()</code>, switch on/off a probe) it is possible to process monitored information, e.g., to provide information with the right level of granularity for further tasks (typical examples: averages, min, max over a stream of information/data) reason on the monitored information wrt whether the system behaves as intended; optionally including filtering; all other analysis activities (statistical analysis, prediction, diagnosis, etc.) including required persistence are out of scope of this work! filtering/checking at runtime, while the monitored system is operating, live filtering/checking performed at defined points in time or based on certain defined (time-based) conditions filtering/checking performed as soon as possible (e.g., triggered by events/event data) manually or when certain (non-regular time) condition holds true, e.g., when the CPU load is low or when the monitoring engineer sees a need filtering/checking post-mortem based on recorded event logs

Table 3

Domain model (3/5): constraint/condition definition and reasoning.

Domain model element (ID)	Definition/Example
* <i>How are conditions/constraints defined (CCD)</i> · Time of definition · development time · runtime · Support to define conditions/constraints (cf [4].) * <i>Trigger (TRI)</i> (cf [4].) · periodical · continuous · on demand * <i>Language (LNG)</i> · Expressiveness · Boolean · Numbers · Quantifiers · Temporal · Fuzzy · Combined · Representation · textual/DSL · programming language · graphical * <i>Reasoning (RES)</i> (cf. [4]) · Formal approaches · CEP · Models/Checkers · AI/KB · Proprietary/custom · Other * <i>Where does filtering/checking happen (LFI)</i> · local to process/application · local (same (virt.) machine) · distributed (central) · distributed (network)	when are conditions/constraints defined define conditions/constraints before starting the monitored system define conditions/constraints while the monitored system is running what conditions/constraints can be defined? E.g., invariants, conditions, etc. how is the evaluation of conditions/constraints triggered evaluation triggered based on time conditions, e.g., no matter if new info available, check every x secs evaluation triggered based on monitored information (events/event data), i.e., continuous checking evaluation triggered on demand, e.g., via Tool/User how are conditions/constraints defined what can the language to define conditions/constraints express for all, exists, etc. for a set of data conditions/constraints on events/timing behavior uncertainty, fuzzy constraints any combination of one of those above what representation is used to define conditions/constraints what mechanism is used to check conditions/constraints e.g., SAT, automata, constraint and model checking complex event processing e.g., UML, Goal Models artificial intelligence or knowledge-based where does the reasoning mechanism run (logically) in the same process/application local, but separated from the monitored application done on one separate node in the network spread over multiple nodes in the network

domain model—authors of this paper not involved in the initial creation of the reference architecture, represented the architectures of their own monitoring approaches as instances of the reference architecture. The results are presented in Section 7. Feedback from this exercise was incorporated in terms of a refinement of the reference architecture. Adap-

tations of the reference architecture of course also led to corresponding adaptations in the instances.

Furthermore, with the help of experienced developers, who are not authors of this paper, we created another instance of the reference architecture for the Kieker approach [10,25]. This exercise further confirmed

Table 4

Domain model (4/5): collection information, application context, target language.

Domain model element (ID)	Definition/Example
Collected information (CIN)	what (types of) information are collected by the approach
- Resources * CPU * Memory * Network * File System * Energy * Others - Events/Messages * Objects/Data - State Info - Other - Data Definition/Types * no (application) data collected, just events/messages * pre-defined * self-defined (flexible) * Types supported · Boolean · Numeric · Strings · Arrays · Lists · Others * Data volume is constrained	e.g., Processes/Threads, Response Time,... in the simplest case a timestamp + a name; can be related with resources; can have diverse attributes data items attached to events, e.g., sensor values, parameters information on the state of a system or component of the monitored system, e.g., maintenance mode, initializing,... how is the data attached to events or describing resources defined if this is the case, the fields below become irrelevant there is a minimal list of pre-defined data types possibility to define new types on top of basic types approach sets a limit on volume of data that can be collected
Appl. Context (ACO) (cf. [4].)	categories of supported monitored systems
- embedded * hard real-time * soft real-time - information systems - large-scale/distributed/service-oriented - large-scale/large-volume	basically any non-embedded system service-orientation is explicitly mentioned or the scale / distribution of the system explicitly addresses systems that deal with big data, e.g., Hadoop, Apache Storm, Apache Spark
Target language (TLA)	languages of monitored systems that can be monitored
- Java - C .NET-based languages - unrestricted/out of scope - Other	approach explicitly unrestricted or no language mentioned

Table 5

Domain model (5/5): target platform, architectural style, intrusiveness, inputs, outputs, tool support, variability, validation, availability.

Domain model element (ID)	Definition/Example
Target platform (TPL)	systems that can be instrumented/monitored
- Java VM or J2EE .NET - Native - Operating System - Service-oriented system - unrestricted/out of scope - Other	e.g., Mobile (Android, IOS); embedded (QNX, RTOS, MicroSAR OS); Linux, MacOS, Windows explicitly defined as unrestricted or no target system mentioned
Architectural style of monitored system (ARC) (cf. [4])	what type of architecture (see [68]) is the target of the monitoring approach?
Intrusiveness (ITR) (cf. [4])	effect of the monitoring solution on the monitored system (potential performance overhead [69])
Inputs (INP) (cf. [4])	what is the basis to apply the monitoring solution to a system, e.g., requirements specifications, SLAs, models, etc.
Outputs (OUT) (cf. [4])	what is the monitoring solution's output, e.g., violations, system state informations, etc.
Tool Support (TSU) (cf. [4])	an interface to the user of the monitoring solution, e.g., an end-user tool or a command-line tool
Variability (VAR) (cf. [4])	(re-)configuration of the monitoring solution beyond initial set-up and configuration
Validation (VAL) (cf. [4])	evaluation of the monitoring solution, e.g., by examples, empirical, formal, etc.; see [70]
Availability of any tools (AVT) (cf. [4])	are any tools downloadable? How are they provided? Are they still maintained?

the applicability of our reference architecture and led to one minor refinement.

Finally, we documented the whole study in terms of this publication. While many steps in our research method rely at least in part on our earlier experiences and background knowledge, we believe that the iterative refinements performed following the many discussions among the authors of this paper (coming from different fields), and particularly the results of applying our work to existing monitoring approaches give our results credibility.

4. Domain model

We use the term domain model similar to Lim [53] and Kang et al. [54], where a *domain model describes the requirements relevant to characterize all systems in a domain*. According to Lim [53] a “domain model represents the requirements in the domain by describing the functions, constraints, and structure within the domain”. Here, the domain under study is the domain of monitoring systems. Of course, it can not be guaranteed that all possible requirements are covered. Rather, it is generally

understood that scoping needs to be performed to define an appropriate range of the domain [55]. For our domain model, we addressed the scoping problem from two sides (cf. Section 3): first we used an intentional approach by (bottom-up) creating an initial domain model, which we then validated—and extended, where appropriate—by comparing it against a large number of actual monitoring systems found in literature.

Our domain model is described in Tables 1–5. In this section, we define each element of the domain model and provide examples. Some elements have also been discussed in our earlier work on a comparison framework for runtime monitoring approaches [4]. We highlight these elements in the tables with an explicit reference to the earlier publication. Elements that have informed the development of our reference architecture (cf. Section 6) are highlighted in italics and blue color. We define unique IDs that we use when referring to the model in the remainder of the paper and also in the online spreadsheet [26].

As shown in Table 1, we augment the model also with categorization information like the *name* (NAM) and *purpose* (PUR) of the approach. For instance, an approach might focus only on resource monitoring [11], solely on requirements monitoring [18], or support both areas [23,37,56,57]. Some monitoring approaches might have purposes that can neither be considered resource nor requirements monitoring, e.g., we encountered cross-layer monitoring of multi-cloud service-oriented architecture apps [58], and architectural visualization and call analysis [30]. Additionally, monitoring approaches may support adaptation [56] or evolution [10]. As this categorization information can not actually be considered as elements of the domain, we visually separate name and purpose from the rest of the domain model in Table 1.

The domain model describes the *life-cycle support* (LCS) a monitoring approach provides, i.e., to which development phases of the monitored system the approach contributes, e.g., *requirements engineering* [9], *design* [13], *implementation* [59], *testing* [38], *commissioning* [18], *operation*, and/or *evolution* [41].

Another key part of the domain model are the *supported phases* (SPH) of the monitoring process. We consider the following main phases (see Tables 1–3). Together they describe all activities relevant to monitoring.

- **Scoping (SCO):** defining **what to monitor** and **from where**? Possible scopes are: hardware, software component or service, platform, runtime environment, system, and/or relation.
- **Collection of monitoring information (COL):** defining **how to collect monitoring information** such as events or in particular event data. This includes the instrumentation process, which can either exploit third-party tools [11] and, therefore, be completely transparent to the user of the monitoring approach [60]; or be explicitly supported via specific tools or languages for probe insertion. Code might be changed manually [36] or using techniques such as AspectJ [30]. Probes can be manually generated, using a programming language [18], or be defined declaratively and then automatically generated through model transformation [111]. Our domain model also allows us to capture **when monitoring information is collected**, e.g., periodically [43]; continuously, which is the case for most approaches [61]; or on-demand [23].
- **Aggregation (AGG):** in this phase the monitored information is manipulated to provide information with the right level of granularity for further tasks, e.g., by calculating averages, minimum or maximum values over a stream of events. As a further example, the approach by Zmuda et al. [62] calculates the invocation rate—i.e., the rate at which service requests respond successfully—based on the service requests that are sent and the service responses that are received. Another example is ReqMon [9], which can calculate averages, sums, and several other functions based on monitored events and data attached to events.
- **Filtering/Checking (FIC):** in this phase we devise **how to reason** on the monitored information, to understand whether the system behaves as intended. This may optionally include filtering. Other analyses, such as statistical analyses, prediction, diagnosis, etc., are out

of the scope of this element. Filtering/Checking includes information about:

- When the filtering or checking happens (TFI). They may happen either online (periodically [63], continuously [9], on demand [62]) or offline [38] (post-mortem based on recorded event traces).
- How the conditions or constraints are defined and when (CCD). Invariants [11] as well as pre-conditions and post-conditions [17] are possible types of conditions or constraints. They can be defined at development time [64] or runtime [18].
- How the evaluation is triggered (TRI). Evaluation can be triggered periodically [65], continuously [66], or on demand [67].
- How the conditions or constraints are defined (LNG). They can be hard-coded in an existing programming language such as Java [67], or written in a declarative domain-specific language (DSL) [9]. Furthermore, the languages can have different expressive powers and representations. For example, they can be defined using a textual DSL [18], a programming language [67], or graphically [28].
- What mechanisms are used to check conditions or constraints (RES). For instance, formal approaches such as SAT solvers [28] or automata [66] can be used, or a complex event processing engine such as Esper [23]. Model checking techniques [28] can be applied or the approach may use its own custom reasoning engine [18].
- Where filtering or checking happens (LFI). They may be performed local to the monitored process or application [62], local in the same (virtual) machine [12], distributed on a separate node in a network [18], or distributed and spread over multiple nodes in a network [58].

The domain model further includes details regarding the *collected information* (CIN) (cf. Table 4), i.e., what (types of) information is collected by the monitoring approach. Resource monitoring approaches can, for instance, collect information about CPU [71] and memory [12] consumption, network traffic [40], file system usage [72], energy consumption [73], as well as information regarding processes or threads [12]. Requirements monitoring approaches, on the other hand, typically collect events, potentially with objects or data attached to them [9,18,23]. Some approaches might just collect state information [36], i.e., information on the state of a system or of a component of the monitored system, such as “maintenance mode” or “initializing”. Data attached to events can exploit pre-defined data types (Boolean, numeric, etc.) and/or use self-defined/flexible types (like in object-oriented programming). Some approaches might also be constrained when it comes to the data volume they can handle; however, we did not find information about that during our analyses (cf. online spreadsheet [26]).

As shown in Table 4 and Table 5, the model further describes the *application context* (ACO), e.g., embedded systems [36], information systems [35], or large-scale/distributed systems [18]; the *target language* (TLA), i.e., the development languages used by the systems that can be monitored, such as Java [12], C [36], .NET-based languages [60], etc.; and the *target platform* (TPL), i.e., the systems that can be instrumented and monitored, such as Java VM-based [12] or service-based systems [23]. In contrast to the application context, the target platform focuses specifically also on the technology of the monitored system, e.g., Java VM, .NET, concrete (type of) operating system.

Additionally (cf. Table 5) – similarly to and inspired by our comparison framework for runtime monitoring [4] – our domain model captures information about:

- The *Architectural style of the monitored system* (ARC): what type of architecture is the target of the monitoring approach, e.g., service-oriented architecture [34] or cloud-based [58]?
- The approach’s degree of *Intrusiveness* (ITR): what is the effect of the monitoring solution on the monitored system? When probes run

within a monitored system they can generate performance overhead. How is this overhead measured? Some approaches might only listen on an event bus [34], while others might instrument code leading to significant additional resource consumption [30].

- The approach's *Inputs* (INP): what is the basis for applying the monitoring solution to a particular system? For instance, are requirements specifications, service-level agreements, or goal models used as inputs? This will be very specific to the particular approach, but typically revolve around some kind of specification defining what the monitored system is expected to do and/or with what degree of performance.
- The approach's *Outputs* (OUT): what is the output of the monitoring solution? For example, are violations produced or is system state information communicated? Similarly, this will be very specific to the particular monitoring approach but often revolve around detected violations of performance goals or requirements.
- The approach's *Tool Support* (TSU): is there an interface to the monitoring solution? These could be end-user tools or command-line tools. Some approaches come with fully-fledged tool suites, such as Kieker [10] or REMINDS [18], while others have basic command-line tools [62] or no tool support at all.
- The approach's degree of *Variability* (VAR): is (re-)configuration of the monitoring solution supported beyond the initial definitions? For example, Kieker [69] supports the rule-based deactivation of probes.
- The approach's *Validation* (VAL): how was the monitoring solution evaluated? Was a case study with industry performed [10,18] or were laboratory experiments [74] conducted?
- The approach's *Availability of tools* (AVT): Are there any tools available supporting the approach? Are the tools still maintained? Is there an Web site from which they can be downloaded?

To summarize, our domain model defines the key elements of approaches for requirements monitoring and resource monitoring and provides a common terminology to discuss and analyze monitoring approaches.

5. Domain model coverage assessment

In this section we describe how we explored the coverage of our domain model by applying it to 47 different approaches (cf. the online spreadsheet [26]). We also discuss threats to validity.

The 47 approaches cover both resource monitoring (20) and requirements monitoring (27). While four of these approaches were developed by at least one of the authors of this paper, the vast majority of the approaches were developed by other research teams. For one of these [12] we were able to interview the main developer, as he resided at one author's site. The remaining approaches were each assessed by one of the authors of this paper based on peer-reviewed publications. The authors' assessments were then cross-checked by at least one other author, and then discussed pairwise as described in Section 3.

Tables 6–8 describe which elements of our domain model are covered by how many of the 47 approaches we have analyzed, relative to the number of resource monitoring and requirements monitoring approaches respectively, except for descriptive (categorization) elements, i.e., the name and purpose of an approach. We also did not include numbers on the architectural style of the monitored system (ARC), intrusiveness (ITR), inputs (INP), and outputs (OUT), as every approach supports these elements, but in almost as many different variants as we have analyzed approaches. We refer the reader to the online spreadsheet for more details. We decided to summarize numbers on tool support (TSU), variability (VAR), validation (VAL), and availability of any tools (AVT) in Table 8, as not every approach supports these elements.

Please keep in mind that the online spreadsheet is a “living” spreadsheet that is subject to changes, thus minor deviations from the tables in the paper to the online spreadsheet are possible, which is why we

focused on relative and not absolute values and on highlighting big differences.

Specifically, we highlight elements with 0% coverage (in red) and 25% or more coverage (in green). One can easily see that the percentages for many elements are very similar among both areas for most elements, but also that there are some elements with big differences. We have highlighted (in bold font) those fields with 20 percentage points or more difference between both fields. Differences exist mainly with regard to how conditions and constraints are defined and checked (several resource monitoring approaches do not define conditions or constraints at all) and which type of information is collected.

Applying the domain model to these approaches allowed us to show its feasibility and to also to learn about the current state of research in resource and requirements monitoring. A first general observation we made is that *there is only very little overlap regarding the purpose of the respective approaches* in these two fields. 27 of the investigated approaches are requirements monitoring techniques, 20 approaches are resource monitoring techniques, while only four approaches [23,37,56,57]—i.e., 9% of the pool that we considered—represent both requirements and resource monitoring techniques. Another interesting high-level tidbit is that only one fourth of the pool of 47 approaches also cover adaptation. *The majority of the approaches actually focus on identifying interesting or anomalous situations and on communicating them to the monitoring analyst.* This selectivity is also reflected by how the approaches provide support to the different phases of an application's development life-cycle.

In our model we have identified the following seven life-cycle phases: *requirements engineering, design, implementation, testing, commissioning, operations*, as well as *maintenance and evolution*. 30 approaches, i.e., 64% of the pool, support more than one of the identified phases. We have ten approaches supporting two different phases (i.e., 21% of the pool): four support testing and operations, while six support operations and maintenance and evolution. We have five approaches (i.e., 10% of the pool) that support three phases. Four of these approaches support the operations phase: two add support for testing and maintenance and evolution, while one adds support for design and implementation. The fifth approach is slightly different and supports the phases implementation, testing, and evolution. We found four approaches (i.e., 9% of the pool) that support four phases. While all four of them support the phases testing, operations, and evolution, two of them add support for the implementation phase, while the other two add support for the commissioning phase and the design phase respectively. There are no approaches that support five phases, while there is one approach (DynaMICS [64]) claiming to support six of them (all phases except commissioning). Finally, there are no approaches that support all seven of our identified phases. As made evident by our analysis, *the approaches focus heavily on operations, testing, and evolution, either taken individually or in some combination of two or three of them.*

The application of the model also allows us to highlight some interesting findings in terms of (i) the expressiveness of the languages used to define monitoring constraints, (ii) the form of representations used (i.e., textual, graphical, programming languages), and (iii) the form of reasoning employed.

Regarding constraint language expressiveness, a first interesting finding is that only 14 approaches—i.e., 30% of the pool, most of them focusing on requirements monitoring—support quantifiers (e.g., universal or existential). Quantifiers are applied over multiple specimens in a domain (e.g., events produced by multiple sources) or over multiple data points taken from a single event. The data we collected indicates that support for quantification is correlated with support for aggregation, which is to be expected. Indeed, out of the 14 approaches supporting quantification, only three do not support aggregation. A second interesting finding is that 24 approaches, mainly requirements monitoring approaches, support the definition of temporal properties, e.g., for specifying the occurrence of patterns of events over time. This means that *about half of the approaches does not support temporal properties*. Finally, only one approach [60] supports the definition of fuzzy properties.

Table 6

Overview (1/3) which domain model elements are covered by how many of the 20 resource and 27 requirements monitoring approaches we analyzed (details online [26]). We highlight elements with 0% coverage in red and 25% or more coverage in green. We also highlight (in bold font) those fields with 20 percentage points or more difference between both fields.

Domain Model Element (ID)	Resource monitoring approaches	Requirements monitoring approaches
Life-Cycle support (LCS)		
• Requirements engineering	5% (1/20)	15% (4/27)
• Design	0% (0/20)	15% (4/27)
• Implementation	15% (3/20)	15% (4/27)
• Testing	25% (5/20)	30% (8/27)
• Commissioning	0% (0/20)	4% (1/27)
• Operations (Ops)	100% (20/20)	100% (27/27)
• Evolution	25% (5/20)	33% (9/27)
Supported phases (SPH)		
• Scoping (SC0)		
* Hardware	10% (2/20)	7% (2/27)
* SW-Component/Service	85% (17/20)	100% (27/27)
* Platform	30% (4/20)	11% (3/27)
* Runtime	15% (3/20)	4% (1/27)
* System	25% (5/20)	15% (4/27)
* Relation	0% (0/20)	19% (5/27)
• Collection of monitoring information (COL)		
* 3rd party or hidden	30% (6/20)	22% (6/27)
* Probe insertion (manual)	25% (5/20)	22% (6/20)
* Probe insertion (automatic)	70% (14/20)	63% (17/27)
* Probe generation (manual)	40% (8/20)	44% (12/27)
* Probe generation (automatic)	40% (8/20)	44% (12/27)
* When (Periodical/Sampling)	35% (7/20)	11% (3/27)
* When (Continuous)	60% (12/20)	96% (26/27)
* When (On demand)	5% (1/20)	11% (3/27)
• Aggregation (AGG)	65% (13/20)	41% (11/27)
• Filtering/Checking (FIC)		
* When does filtering/checking happen (TFI)		
· Online (periodical)	30% (6/20)	15% (4/27)
· Online (continuous)	25% (5/20)	85% (23/27)
· Online (occasional/on demand)	10% (2/20)	0% (0/27)
· Offline (trace analysis)	25% (5/20)	11% (3/27)
* How are conditions/constraints defined (CCD)		
· Development time definition	55% (11/20)	100% (27/27)
· Runtime definition	40% (8/20)	30% (8/27)
· Support to define conditions/constraints	60% (12/20)	100% (27/27)

Regarding the form of representations used by these monitoring languages our data shows that *there is a strong predominance of textual forms of representation, be them domain-specific languages or (general purpose) programming languages*. More specifically, this is true for 37 approaches, representing 79% of the pool of approaches. Graphical representations, on the other hand, are not used frequently, with only six approaches—i.e., 13% of the pool—all of them from the group of requirements monitoring approaches.

Finally, *regarding the form of reasoning employed by the approaches there is a very strong reliance on proprietary and custom-developed tools*. 23 approaches adopt some form of custom reasoning solution, roughly half of the investigated approaches. This immediately raises the question whether we are continuously re-inventing the wheel. Formal approaches (14) are similarly represented. Finally, *the use of Complex Event Processing techniques is surprisingly low* (eight approaches). This could be due to the technical backgrounds of the researchers, and/or because these techniques have only recently started being adopted outside the field of active databases.

The application of the model has allowed us to also highlight some interesting findings in terms of the application contexts that are currently supported. The first finding immediately standing out is that *research in the fields of requirements monitoring and resource monitoring has*

heavily focused on service-oriented systems. In fact, 25 approaches, i.e., more than half of the pool, focus on service-oriented systems. Without doubt the research in monitoring became a focus for researchers with the advent and growing adoption of service-oriented technologies. In service-oriented systems, engineers no longer have complete control over all the components that are involved in the application they design. Instead, they rely on external (Web-based) services made available by third parties. This introduces a strong focus on trustworthiness, as the functional and non-functional behavior of the external services are out of their jurisdiction. There is a need to “keep an eye” on the services’ correspondence to their expected behaviors. The second finding is that *approaches for embedded systems and Big Data applications are not strongly represented*, with only eight respectively three approaches in our pool. The runtime monitoring of Big Data applications, in particular, seems to be a promising area for future research. Finally, *if we look at the target platforms service-oriented architectures once again prove to represent the lion’s share, followed by Java Enterprise*. Mobile platforms are still not well covered, with only five approaches (of ten approaches targeting operating systems) and, in particular, with no approaches for the iOS platform. This definitely highlights a potential white spot. *If we look at the targeted programming languages, Java is by far the most represented*.

Table 7

Overview (2/3) which domain model elements are covered by how many of the 20 resource and 27 requirements monitoring approaches we analyzed (details online [26]). Coloring and highlighting are explained in the caption of Table 6.

Domain Model Element (ID)	Resource monitoring approaches	Requirements monitoring approaches
* Trigger (TRI)		
· periodical	30% (6/20)	15% (4/27)
· continuous	40% (8/20)	93% (25/27)
· on demand	5% (1/20)	7% (2/27)
* Language (LNG)		
· Expressiveness		
· Boolean	20% (4/20)	74% (20/27)
· Numbers	45% (9/20)	70% (19/27)
· Quantifiers	10% (2/20)	44% (12/27)
· Temporal	15% (3/20)	78% (21/27)
· Fuzzy	0% (0/20)	4% (1/27)
· Combined	10% (2/20)	33% (9/27)
· Representation		
· textual/DSL	25% (5/20)	81% (22/27)
· programming language	40% (8/20)	7% (2/27)
· graphical	5% (1/20)	19% (5/27)
* Reasoning (RES)		
· Formal approaches	10% (2/20)	44% (12/27)
· CEP	10% (2/20)	22% (6/27)
· Models/Checkers	0% (0/20)	7% (2/27)
· AI/KB	0% (0/20)	0% (0/27)
· Proprietary/custom	45% (9/20)	52% (14/27)
· Other	0% (0/20)	4% (1/27)
* Where does filtering/checking happen (LFI)		
· local to process/application	30% (6/20)	22% (6/27)
· local (same (virt.) machine)	25% (5/20)	19% (5/27)
· distributed (central)	35% (7/20)	67% (18/27)
· distributed (network)	5% (1/20)	4% (1/27)
Collected information (CIN)		
• Resources		
* CPU	55% (11/20)	15% (4/27)
* Memory	65% (13/20)	19% (5/27)
* Network	25% (5/20)	15% (4/27)
* File System	15% (3/20)	7% (2/27)
* Energy	5% (1/20)	11% (3/27)
* Others	50% (10/20)	11% (3/27)
• Events/Messages	35% (7/20)	78% (21/27)
* Objects/Data	15% (3/20)	48% (13/27)
• State Info	15% (3/20)	30% (8/27)

Validation is mostly achieved only through examples in the investigated approaches, with resource monitoring approaches focusing a lot on measuring their overhead. There are very few case studies involving real-world systems in realistic execution contexts. It seems that many researchers have not (or could not) make the effort of taking their tools beyond the prototype level, often making them inadequate for real-world applications. The lack of a real-world evaluation highlights possible limitations regarding an approach's performance and/or scalability, making it harder to assess its suitability for a given context.

Similarly, only 25 approaches mention tool support, and even fewer approaches actually make their tools publicly available. Even if they do, sometimes the tools are no longer maintained and supported. This makes it hard to use the approaches in applications that use newer versions of the targeted platforms and programming languages. Furthermore, the absence of tools hinders the actual replicability, or extension, of any evaluations that are presented in the paper. Finally, the absence of tools makes it harder to preserve any valuable knowledge that may have emerged during the development and evaluation of an approach.

Summary and threats to validity

We conclude that although a lot of useful research has been published in both analyzed fields, it might be hard to preserve the accumulated knowledge, especially due to the lack of well-supported and well-documented tools. This can be unfortunate as researchers will need to start investigating how monitoring approaches need to be adapted in new emerging fields, such as Big Data or Internet-of-Things systems. We hope our domain model will be able to facilitate better reuse as it provides a means to at least discuss the existing approaches in the two communities using a common set of terminology.

When it comes to the threats to the validity of our research, the development of our domain model was to a large extent influenced by the 47 approaches we analyzed, and in particular by the information reported in the scientific publications. Specifically, some papers lack details about certain capabilities and also do not discuss possible relevant limitations. Fully assessing all approaches would thus require the involvement of the original developers of the approaches. As our intention was not to provide a complete picture of all existing approaches, but rather to assess the coverage of our domain model for a large number of

Table 8

Overview (3/3) which domain model elements are covered by how many of the 20 resource and 27 requirements monitoring approaches we analyzed (details online [26]). Coloring and highlighting are explained in the caption of Table 6.

Domain Model Element (ID)	Resource monitoring approaches	Requirements monitoring approaches
• Data Definition/Types * no data collected, just events/messages * pre-defined * self-defined (flexible) * Types supported · Boolean · Numeric · Strings · Arrays · Lists · Others * Data volume is constrained	15% (3/20) 35% (7/20) 20% (4/20) 20% (4/20) 40% (8/20) 20% (4/20) 20% (4/20) 15% (3/20) 0% (0/20) 0% (0/20)	22% (6/27) 44% (12/27) 26% (7/27) 44% (12/27) 44% (12/27) 41% (11/27) 33% (9/27) 33% (9/27) 4% (1/27) 0% (0/27)
Appl. Context (AC0) • embedded (hard real-time) • embedded (soft real-time) • information systems • large-scale/distributed/service-oriented • large-scale/large-volume	0% (0/20) 10% (2/20) 45% (9/20) 70% (14/20) 15% (3/20)	11% (3/27) 11% (3/27) 37% (10/27) 67% (18/27) 0% (0/27)
Target language (TLA) • Java • C • .NET-based languages • unrestricted/out of scope • Other	75% (15/20) 5% (1/20) 5% (1/20) 25% (5/20) 5% (1/20)	30% (8/27) 4% (1/27) 7% (2/27) 48% (13/27) 4% (1/27)
Target platform (TPL) • Java VM or J2EE • .NET • Native • Operating System • Service-oriented system • unrestricted/out of scope • Other	60% (12/20) 5% (1/20) 10% (2/20) 25% (5/20) 45% (9/20) 10% (2/20) 5% (1/20)	19% (5/27) 4% (1/27) 4% (1/27) 19% (5/27) 59% (16/27) 19% (5/27) 0% (0/27)
Tool support (TSU)	60% (12/20)	48% (13/27)
Variability (VAR)	30% (6/20)	19% (5/27)
Validation (VAL)	65% (13/20)	48% (13/27)
Availability of any tools (AVT)	45% (9/20)	44% (12/27)

different approaches, we still think analyzing the key publications of 47 approaches is a valid approach.

Regarding internal validity, misinterpretations are always possible when analyzing and interpreting papers, particularly due to the lack of a common terminology and the different styles of (types of) papers in the investigated research communities. We tried to mitigate this threat with our cross-checking and a pairwise discussion phase (cf. our research method). However, we are aware that our analyses and interpretations may have resulted in some misinterpretations. Therefore, we provide our results via an online spreadsheet [26], to allow the developers of the approaches to comment on the information we extracted for their approach and to suggest changes, and for readers to check our analyses in more depth.

Another threat to validity is the selection of approaches and papers for our analyses. However, our selection was based on an earlier systematic literature review on requirements monitoring [5], the development of a comparison framework for runtime monitoring approaches [4], and our own earlier experiences in different monitoring communities [11,17,18]. We are thus confident that the included studies are a representative subset of monitoring approaches. We are well

aware that there are more existing approaches, and thus we intend our spreadsheet to be incrementally extended with new approaches over time.

6. Reference architecture

Based on the findings and insights gained during the development and evaluation of our domain model (Sections 4 and 5), we also developed a reference architecture for software monitoring solutions. The purpose of this architecture is *not* to represent the real architectures of all the 47 analyzed monitoring systems. This would not be feasible as, for most of them, detailed architectural information is not available.⁵ Rather, our goal is to show that the various monitoring approaches exhibit significant conceptual overlap, even though the field is so diverse that no single functionality is part of every monitoring system. Thus, we present a reference architecture supporting the conceptual definition of new architectures. The existing architectures are highly diverse

⁵ We, however, considered other available monitoring architectures, e.g., Robinson [9].

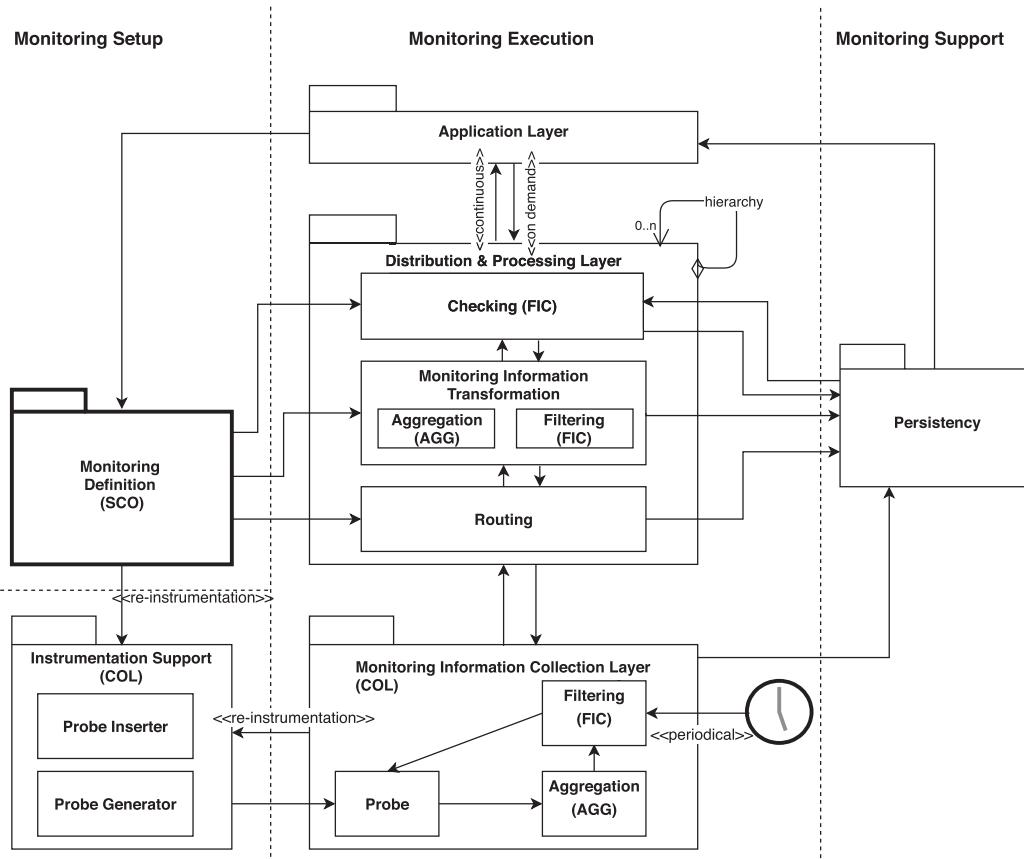


Fig. 2. Reference architecture for monitoring systems.

in terms of the supported functionality. Hence, in our architecture, the only common component is the *Monitoring Definition*. And even for this, the scope of implementation varies depending on the other parts of the architecture. All other components are optional. Before we discuss the various border cases, we first introduce the reference architecture itself, which we created iteratively as described in Section 3.

The reference architecture aims to provide an overview of the capabilities and building blocks of monitoring tools, and the interactions among its parts. We use the UML (Unified Modeling Language) to describe the architecture. As we discuss below, there is a large range of variability inherent in the architecture. Since UML does not provide particular mechanisms for describing variability, and as we did not want to overload the graphical representation of the architecture, we will describe the variability aspects textually.

Our reference architecture (see Fig. 2) can be best understood in terms of three pillars: the left pillar describes the *monitoring setup*. The middle pillar addresses the *monitoring execution*, and consists of several layers. Finally, the right pillar contains *supporting functions*, e.g., for the persistence of monitored information. We discuss each of these pillars in detail and provide references to the domain model to explain the reasoning behind this architecture.

6.1. Monitoring setup

The monitoring setup consists of the *Monitoring Definition* component and the *Instrumentation Support* component.

The monitoring definition specifies what, and how, the system should be instrumented, i.e., what kind of information should be observed. It also provides information on where the observed information should be delivered to, and how additional processing (e.g., aggregation or filtering of information) should be performed. Finally, the monitoring

definition may include rules or constraints to be checked. The different parts of a monitoring definition are represented by the various outgoing arrows. For a specific monitoring solution, however, not all of these parts must be present. Rather, the individual approaches vary in such a way that no individual component (except for the monitoring definition as an abstract concept) is present in all monitoring approaches. As the monitoring definition is the only element present in all specific monitoring definitions, we emphasized it using a thicker frame. In practice, most approaches significantly overlap in terms of functionality of the monitoring definition. Within the limits of a specific monitoring approach, the monitoring definition describes which elements from the domain model (cf. Tables 1–5) are used in a specific instance. A monitoring approach may provide domain-specific languages to define, modify or extend probes; this is, for example, supported by the Kieker languages [75]. There can also be variations for different parts. While a monitoring approach may support the modification of the items to be monitored at runtime, probe insertion may only be supported at development or initialization time.

Based on the monitoring definition the actual probes are defined and inserted into the system implementation (*Instrumentation Support*). Some approaches use *Probe Generators* to generate code for particular types of probes [10,11,40–43,65,74]. In other cases, only probe insertion is supported, e.g., through a *Probe Inserter*; while the probes need to be written manually [30]. Some approaches do not provide any instrumentation support and simply expect the developer to provide and insert all the relevant probe code [76]. Some approaches may also support multiple strategies for collecting monitoring information (cf. Table 2). In this case, the applied approach is provided in the monitoring definition. Finally, the monitoring approach may support different binding times, i.e., allow changes during development, initialization, or at runtime (or several of those).

6.2. Monitoring execution

The core part of the monitoring reference architecture is represented by the second pillar, which is responsible for performing the actual monitoring. It consists of three layers: the lowest layer (*Monitoring Information Collection Layer*) is responsible for collecting the information. The middle layer (*Distribution and Processing Layer*) is responsible for transporting and analyzing the information. The top layer (*Application Layer*) contains visualization functionality and/or other application-specific capabilities, often regarded as outside the scope of the core monitoring approach.

Probes are the source for any type of information collection, and they exist in many different kinds and facets. For example, in the context of resource monitoring, probes might be directly responsible for collecting CPU or memory loads [10,11,40–43,65,74]. For monitoring tools that focus on environment monitoring, these might be directly connected to external sensors. In the context of requirements monitoring approaches [13,18,34,60], probes often collect system events and monitored information, e.g., the system components being called, variables being modified, etc.

A few monitoring approaches are also capable of performing (re-)instrumentation at runtime. This can be triggered by the *Monitoring Information Collection Layer* — because a probe is found to be inappropriate — or by a higher layer. We represent this with a connection in the diagram between the *Monitoring Information Collection Layer* and the *Instrumentation Support* component. This connection is, however, only present, if *re-instrumentation* is supported as denoted by the stereotype. In this case, instrumentation needs to be possible at runtime (hence, in some cases instrumentation support must be available during both monitoring setup and monitoring execution). To denote this, we also added a horizontal line, with the stereotype *re-instrumentation* support.

Sometimes aggregation and filtering are required as part of the information collection as described in the domain model. *Aggregation* might be needed to convert the information into an adequate form, either because passing all the information for further processing would impact the performance of the monitoring/monitored system or because the required bandwidth would exceed the (physical) limitations on processing and transportation. For example, one may want to know how much memory is used during a certain (method) call, including all dependent calls. The only way to avoid excessive overhead is then to collect all the individual memory allocations until the call completes, and to perform an early aggregation to add them up [11].

The *Filtering* component is responsible for performing data selection. Filtering can happen in one of two ways: either by discarding all the information that is actually not required, or by only collecting information when it is needed in the first place. The latter is illustrated by the backward relation between filtering and the probes on the monitoring information collection layer. As described in the domain model in Section 4, the collection of information can happen periodically, continuously, or on demand. In case of periodical collection, information is passed on by the filter component at certain timed moments, illustrated by the clock icon in Fig. 2. No filtering happens for continuous information collection, as all acquired information is just passed on as soon as it is available. Finally, on-demand gathering is represented by the downward arrow that enters the information collection layer. In this case the filtering component passes on the information once a request is received.

It should be noted that our reference architecture provides two levels on which filtering may happen: within information collection, meaning that the information is already discarded during the acquisition step [11]; and during analysis, which may happen at a different node in the network [18]. Not all monitoring systems provide these two levels. If two levels are supported, typically only very simple conditions are supported for filtering within data collection, while more complex filtering is provided in the analysis layer. In our domain model, this is represented by the element *trigger for evaluation* (which is

independent of the filtering level) and the element *where does filtering/checking happen*.

The aggregated and/or filtered information is passed on to the *Distribution and Processing Layer* as shown by the upward arrow. This layer consists of three main components: *Routing*, *Monitoring Information Transformation*, and *Checking*. These elements are further defined by the domain model elements *Trigger for evaluation*, *Language*, and *Reasoning*.

Often monitoring approaches themselves are distributed, e.g., checking of information happens on different nodes than the ones that are responsible for gathering the information [18,24]. The purpose of the routing component is thus to bring data gathered at some node to another node, which will then be responsible for further processing the information. The routing component will transfer all the information passing the filtering stage of the monitoring information collection layer. The monitoring information may then be further aggregated and filtered. The information resulting from the transformations is then passed on to the checking component. The capabilities of the checking component may vary widely, especially if temporal constraints are checked. Furthermore, local buffering of information may also be supported.

In the case of requirements monitoring approaches in cloud-based systems the complete analysis layer, including the routing component, may be iterated, successively transferring monitoring events to higher-level nodes where these are aggregated and filtered [77]. However, in practice, even in approaches that support a hierarchical topology, not all components of the analysis layer are replicated on all involved nodes. Finally, the processed and checked information is forwarded to the *Application Layer*, often in the form of checking events like alarms signalling the violation of certain boundaries. In our reference architecture, visualization and representation of the final information is regarded as just one form of application interested in performing some form of further processing. In principle, an application may change the monitoring definition, which is described by the corresponding arrow.⁶

6.3. Monitoring support

The final pillar of our reference architecture is monitoring support. Its main role is to provide *Persistence*. Like the other parts of the architecture, this is not provided by all monitoring frameworks, and may take very different forms, ranging from simple trace files to entire database solutions. In recent years, several initiatives have pushed on standardization for persistence and interchange formats, e.g., Open.XTrace [44] or OpenTracing⁷ for performance monitoring.

Information for persistence is forwarded by the processing component. Typically, this is filtered and aggregated information, although checking results are also forwarded (e.g., alarm notifications). Some approaches also directly support persistence of the raw information that is provided by the information collection layer [18]. This can happen locally or remotely. In Fig. 2 this is reflected by the outgoing arrows from the routing component to persistency (for remote storage) and directly from the information gathering layer (for local storage).

Depending on the expressiveness of constraints, approaches also support the use of historical data as part of checking. This is modeled as a backward arrow from persistency to checking. Such use of data can happen during runtime or in the form of an offline analysis. However, offline analysis is often done in the form of specific applications analyzing the collected trace information in ways that are different from the checking criteria defined as part of the monitoring definition. This is represented by the arrow between persistency and the application layer.

6.4. Monitoring variability

We consider the architecture shown in Fig. 2 a reference architecture, as it subsumes a large range of different specific architectures. This

⁶ This was introduced based on the Kieker case study (cf. Section 7.4).

⁷ <http://opentracing.io/>.

variability is shown by the fact that all components (except for monitoring definition) as well as their relations are optional in concrete instances of a monitoring system. Also, the details of the functionality of the individual components may vary widely as discussed above and described by the domain model. Also, the hierarchy given by the corresponding relation in Fig. 2 can be instantiated in different ways.

As stated earlier, the goal of defining the reference architecture was not a replication of existing specific architectures, but rather, to provide a reference structure that can easily be instantiated to represent the behavior of existing monitoring systems. As a consequence, we designed the architecture in a way that it allows to easily represent the variation. On the highest level this led to the structure of the pillars and the different layers within them. We experimented here with different structures and then picked these layers, as they provide meaningful abstractions and make it easy to represent relevant variations (by de-/selecting the layers).

One example is the depicted hierarchy. As some approaches allow for hierarchy, we decided to make the hierarchy the general case as it subsumes the more common cases. Non-distributed monitoring systems may then simply have the processing layer on the same machine as the monitoring information collection layer.

We also aimed at unifying the aggregation and filtering in both layers, but found it hard to adequately represent this in a unified way due to their slightly different purpose. In the collection layer, aggregation and filtering leads to data generation, while on the processing layer, such events are already the input.

Below, we will briefly illustrate a couple of extreme cases that would still be considered variants of our reference architecture to illustrate the large range of implied variability.

Our first (artificial) extreme example is a pure resource monitoring system only collecting resource information, without performing any further processing. All the information is written to a log/trace file for offline analysis. Because it (only) supports a set of pre-defined resources, it is capable of automatically generating the probes and inserting them into the monitored system. This monitoring system would actually make only use of the lower part of the components (and relations). The monitoring definition would identify the resources to be monitored and the necessary locations for inserting the probes in the monitored system. In instrumentation support, both the probe inserter and the probe generator would be used. The probe generator would create the probe and the inserter would instrument the code of the monitored system accordingly. At runtime the probes would produce the relevant information; since we assume that no further filtering and aggregation is done, the information would be continuously forwarded to the persistency component, which would store it in a trace file.

Our second (artificial) extreme case is a distributed requirements monitoring system, which only supports the analysis of the information and the forwarding of alarms to an application. In this case, there would be no information collection layer; rather, the developers of the monitored system would have to separately implement it. Thus, the monitoring system would not contain instrumentation support or gathering of information. A monitoring definition would exist, however, and focus on routing the information (e.g., describing which server contains the checking information) and on the checking rules. Thus, its actual content would be complementary to the previous example. The routing may send the information to a processing server, where it is aggregated and filtered, and send violations to the application layer.

The interesting aspect of the comparison of these two extreme cases is that they are complementary and the intersection of the architectural elements is basically empty.⁸ Nevertheless, both cases clearly qualify as software monitoring approaches, according to typical definitions of

monitoring in software systems. We will further evaluate the defined reference architecture based on a number of real monitoring systems in the next section.

Summary

In this section, we presented a reference architecture which captures the full breadth of functionality supported by the domain model described in Section 4. It should be kept in mind that it is *not* the goal of this architecture to represent the full variability exhibited by actual monitoring architectures. Rather, it provides a canonical representation of this functionality. However, we believe, it could also be well used as a blueprint in the construction of future monitoring systems.

7. Reference architecture evaluation: instantiation for different monitoring approaches

In this section we describe how we evaluated our reference architecture by instantiating it for five different monitoring approaches. We first instantiated it for the four approaches we know best, those we developed: SPASS-meter [11] (with and without EASy-Producer [78] integration as used in the QualiMaster solution [24]), REMINDS [18], and ECOWARE [23]. For these four selected instances, the authors possess the required detailed knowledge to perform and discuss the mapping. However, this incurs a certain risk for bias. In order to mitigate this bias, we also instantiated the reference architecture for Kieker [10,69] with the help of some Kieker experts.

7.1. SPASS-meter

First we instantiated our monitoring reference architecture for SPASS-meter. We divide the discussion into two parts: (i) The SPASS-meter resource monitoring framework [11] as a standalone tool and (ii) the integration of SPASS-meter into the QualiMaster-solution to enable requirements monitoring for a distributed architecture along with monitoring for adaptation [24].

7.1.1. The SPASS-meter framework

SPASS-meter⁹ [11] is a resource monitoring framework for Java programs and Android Apps. It aims at flexibility, online information aggregation, and low monitoring overhead for resources including CPU/execution time, memory allocation, network-I/O, file-I/O and battery consumption (for certain platforms).

The processing flow of SPASS-meter starts with defining the monitoring scope. The monitoring scope definition can be given as source code annotations within the monitored system, as an (external) XML-file for legacy systems or in a mixed fashion combining annotations and the XML file. A monitoring scope defines the relevant resources on operating system, JVM and monitoring group level. A *monitoring group* is the most basic aggregation unit representing an arbitrary part of the monitored system, e.g., a service, given in terms of classes or methods. Based on the scope, the monitored system is instrumented, either before (static instrumentation) or at startup time (dynamic instrumentation) of the monitored system. At runtime of the monitored system, raw resource consumption is collected and aggregated to the defined monitoring groups or system levels. Finally, the aggregated results are passed to optional presentation plugins for further analysis, e.g., for constraint checking in adaptive systems [78].

Over the last few years, SPASS-meter was applied to different kinds of systems, ranging from Android Apps and service-based systems to distributed Big Data cluster systems. These cases demonstrate the flexibility and openness of SPASS-meter, as each type of system required

⁸ More precisely, the monitoring definition is part of the intersection, although if we look at it in more detail, the information actually contained in both cases would be disjoint.

⁹ SPASS is the acronym for *S*implifying the *d*evelopment of *A*ddaptive *S*oftware *S*ystems and SPASS-meter is a basic building brick of our work on that topic. In German, “Spass” means “fun” and points to the tons of fun the authors had while realizing the framework.

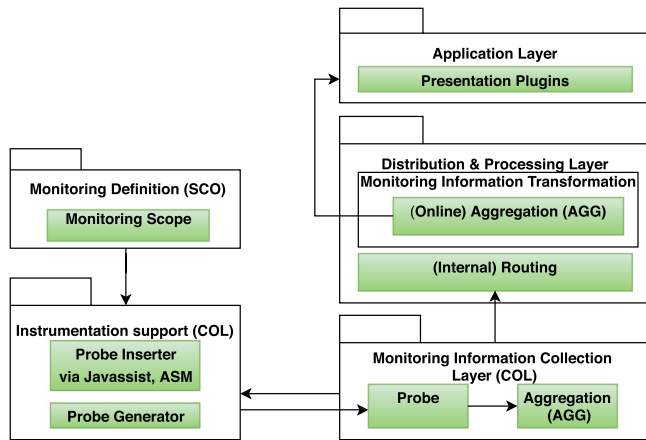


Fig. 3. Instantiated reference architecture for SPASS-meter.

different aggregation semantics (e.g., for components or services [11]), data analysis algorithms [24], and benchmark tasks [79]. The architecture of SPASS-meter [11] follows this processing flow.

Fig. 3 illustrates the components of the SPASS-meter framework in terms of the reference architecture. The *Monitoring Definition* component in our reference architecture is instantiated by the SPASS-meter monitoring scope. As SPASS-meter does not include a checking mechanism, it does not support any checking information (such as constraints) in the monitoring scope definition. The *Instrumentation Support* takes the monitoring scope definition and instruments the monitored system accordingly. For the instrumentation process, the *Probe Generator* defines instrumentation strategies for the various resources supported by SPASS-meter. The *Probe Inserter* turns the instrumentation strategies into bytecode for static or dynamic instrumentation. For experimental purposes, SPASS-meter provides two alternative probe inserters, one based on the Javassist bytecode framework¹⁰, which operates based on Java code fragments given as Strings, and one based on ASM¹¹, a low-level bytecode manipulation library. To achieve better performance and to further reduce the overhead, SPASS-meter can dynamically re-instrument code utilizing native JVM support, i.e., by adding or removing resource-intensive memory-usage probes [11]. Therefore, the instrumentation support is also available at runtime.

At runtime the inserted probes provide raw resource consumption information to the *Monitoring Information Collection Layer*. The probes also indicate the points in time in which the control flow enters or exits a monitoring group. Due to performance reasons some raw information such as memory allocations are already aggregated during probing when exiting a monitoring group.

Usually, in SPASS-meter all instantiated layers of the reference architecture are executed on the same physical machine as the monitored system. If beneficial for the overhead/performance, SPASS-meter can be executed in client-server mode so that monitoring information collection is performed on the machines running the monitored system, while aggregation and/or presentation happens on separate machines. In client-server mode, transport and presentation can also be executed on a different machine to reduce the overhead on the monitored system. The client-server mode is mainly realized by the internal routing mechanism and can be set up based on information in the monitoring scope. Further, the *Distribution & Processing Layer* also contains the online aggregation component, which assigns the raw monitoring information to monitoring groups. Finally, in the *Application layer*, the collected information is passed to the registered (optional) presentation plugins.

Presentation plugins can be added based on the needs of the monitoring situation, e.g., to perform further analyses.

7.1.2. The QualiMaster solution

SPASS-meter allows collecting information about the resource consumption of a program, but it does not provide capabilities for constraint checking. The QualiMaster-project¹² (QM) developed a resource-adaptive, distributed data processing infrastructure, which uses constraint checking as a basis for runtime adaptation. To avoid re-implementing existing components in a project-specific manner, this was built based on existing, proven components: SPASS-meter for monitoring the resource consumption on the compute nodes and the EASY-Producer-tool [78] for centralized constraint-checking of the aggregated resource consumption. These components were connected and integrated using a solution-specific routing component. While other monitoring tools such as REMINDS could also have been integrated for constraint checking we used EASY-Producer because it supports flexible artifact instantiation (including code generation and modification at runtime). This integrated solution is discussed in more detail in [24].

Fig. 4 illustrates the integrated QualiMaster monitoring solution, including components from SPASS-meter ('green') as discussed above, components from EASY-Producer ('blue'), and solution-specific components ('red').

SPASS-meter is used to monitor the resource consumption on the compute nodes of a cluster, i.e., each of the cluster nodes for data processing executes at least one JVM with data analysis algorithms instrumented and monitored by SPASS-meter. The online aggregation of SPASS-meter runs on the cluster nodes, i.e., the client-server mode of SPASS-meter is not active in this solution. A specific SPASS-meter presentation plugin is responsible for sending the collected resource consumption information (along with cluster node identification) twice per second to a centralized aggregator. This is achieved via a solution-specific routing component, the QM event bus, which also provides this information to other QM components and supports their communication. The information sent by the distributed SPASS-meter instances is collected by the central QM aggregator, which determines the actual load and resource consumption of the running data processing applications, the nodes and, in turn, the entire cluster. The QM aggregator is prepared to operate in a hierarchical fashion, however, this capability was not needed in QualiMaster and is thus not indicated in Fig. 4. Once per second, the aggregator maps the aggregated information onto a model representing the potential runtime changes that could be made in such a cluster. This model is part of the *Monitoring Definition* of the solution. Its constraints are given in an OCL-like notation, and predicate over the resources that are utilized at runtime. The models are read and analyzed by EASY-Producer. If any resource constraints are violated, detailed information about the violations are sent to the QM adaptation component in the QM *Application Layer*. The QM adaptation component exploits EASY-Producer functionality to decide the runtime changes that need to be made to the data processing components in the cluster. Thus, EASY-Producer is used both for constraint analysis (checking) in the transport layer, and for adaptation in the application layer.

Summary

Both concrete architectures presented in this section could be represented as instances of the proposed reference architecture. SPASS-meter—as a standalone resource monitoring framework—only instantiates a subset of the components of the reference architecture, similarly to the extreme case discussed in Section 6.4. The integration of SPASS-meter with a constraint checking component, such as EASY-Producer, represents an integration of resource and requirements monitoring and, thus, instantiates the complete reference architecture.

¹⁰ <https://github.com/jboss-javassist/javassist>.

¹¹ <http://asm.ow2.org>.

¹² <http://qualimaster.eu>.

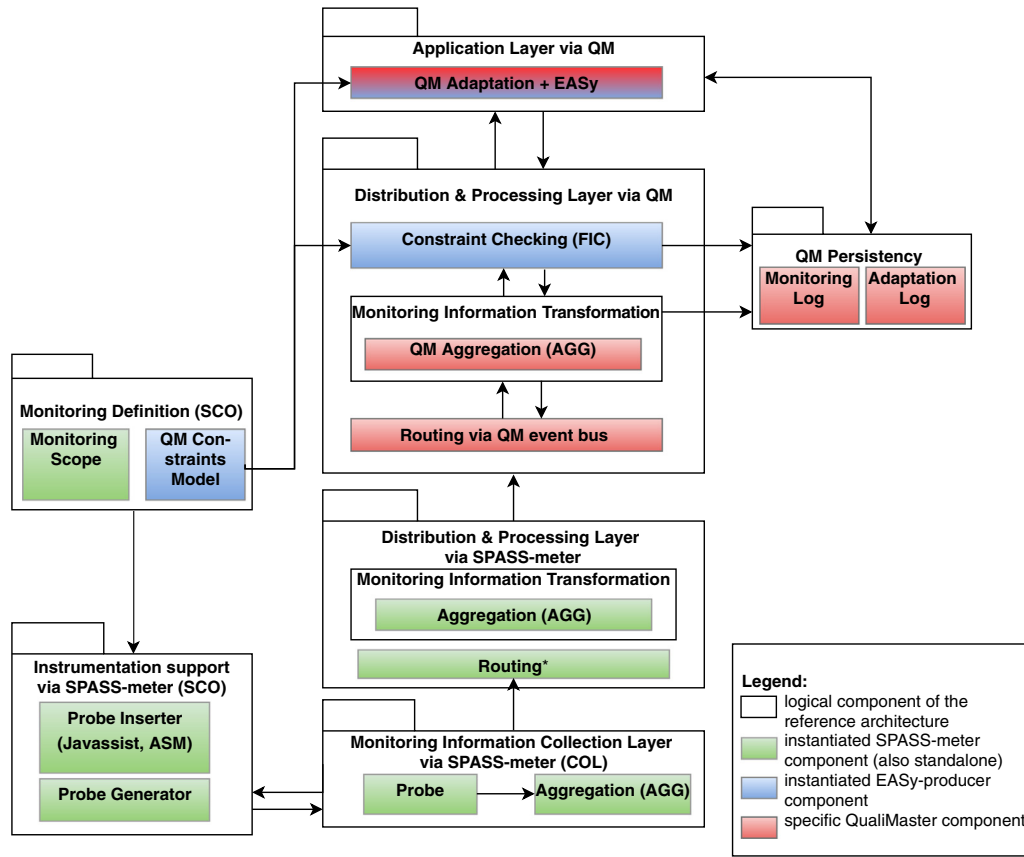


Fig. 4. Instantiated reference architecture for SPASS-meter and EASy-Producer. (For interpretation of the references to color in the text, the reader is referred to the web version of this article.)

The architectures presented in this section also demonstrate that our reference architecture allows for variability. This is needed in real-world settings, where instantiations of the reference architecture are never one-to-one mappings of the components. As seen in the QM solution shown in Fig. 4, the monitoring definition is instantiated by two different specifications due to the integration of the complementing tools SPASS-meter and EASy-Producer. Further, distribution & processing and the application layers of the reference architecture appear duplicated, as SPASS-meter is distributed onto the compute nodes and EASy-Producer is used in a centralized fashion.

7.2. Reminds

Second, we instantiated our monitoring reference architecture for REMINDS¹³ [18], which was developed for monitoring large-scale systems of systems (SoS) and has been applied in the automation software domain. It follows the aforementioned Robinson layers [9], and adds additional support to cope with the specific characteristics and complexity of SoS. Specifically, the REMINDS framework [18] consists of four dedicated monitoring layers. The top and bottom layers of the framework enable the development of arbitrary system-specific probes and clients for different domains and technologies, whereas the two core layers are independent of the actual systems to be monitored and provide basic monitoring capabilities such as aggregating and analyzing monitored events and information.

REMINDS is based on the requirements monitoring model (RMM) for SoS [80], which comprises three different key elements. *Monitoring Scopes* define the areas of interest to be monitored (similar to SPASS-meter) and hierarchically represent the architecture and, to a certain extent, the organizational structure of the SoS. *SoS Requirements* describe functionality, properties, or behavior of the SoS to be checked by REMINDS runtime. Modelers formalize requirements as constraints to be checked at runtime based on events and event data. Typical constraints [81] define expected event sequences, event timing, or allowed ranges of monitored information. Monitoring information such as *SoS Events* are collected by probes that are deployed to the different systems in the SoS using instrumentation. They conform to event models that provide a uniform representation and a common scheme of events and monitoring information. Each constraint in the RMM is related with the events needed to check for violations.

The architecture instance for REMINDS (cf. Fig. 5) covers all layers of the reference architecture, however, not every layer is covered to its full extent.

The *Monitoring Definition* layer comprises components for defining the Monitoring Scope, Constraint Templates, and Constraint Definitions. A scope in the context of REMINDS represents a system (or component) of interest that is to be monitored. Using a dedicated domain-specific language [81], REMINDS provides a simple and easy way to define constraints. Based on several different constraint templates provided by REMINDS, constraints for checking the correctness of a system or component can be created using the previously defined event types. Due to the heterogeneity of the systems that contribute to a SoS, and their diversity in terms of technologies and implementation languages that are used, providing *Instrumentation Support* is quite challenging. REMINDS thus supports various ways of creating probes for

¹³ REMINDS is the acronym for REquirements Monitoring Infrastructure for Diagnostics Systems of systems.

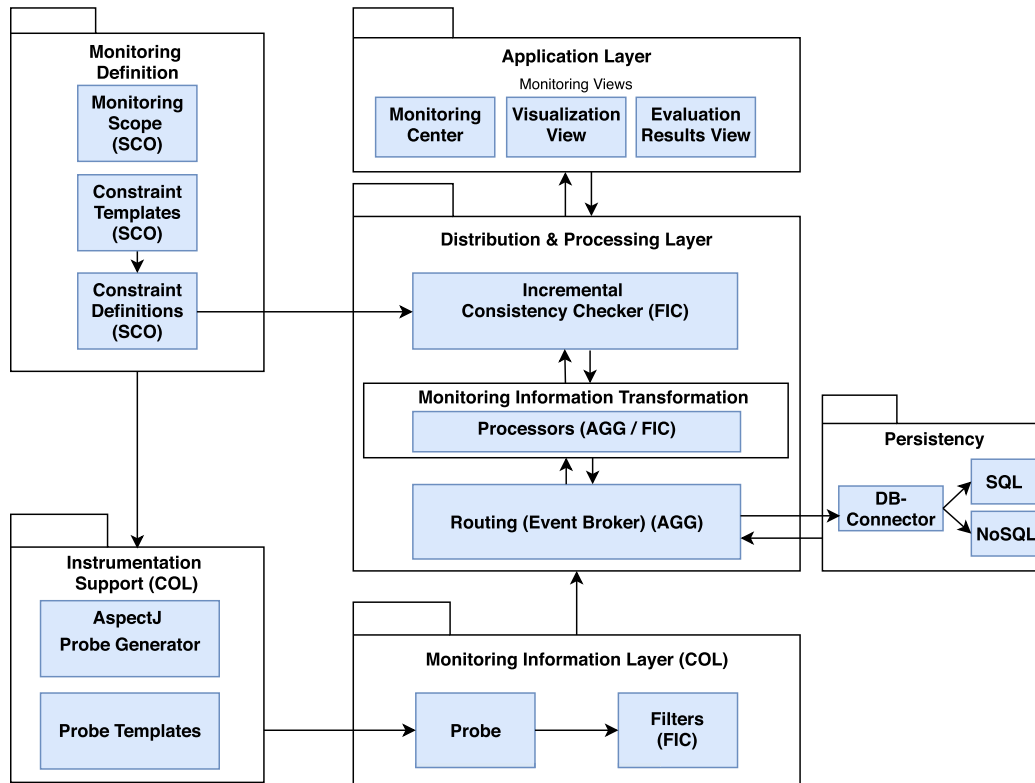


Fig. 5. Instantiated reference architecture for REMINDS.

instrumenting the respective systems. This ranges from manually creating probes and forwarding events to tool support via a predefined interface, to automatically generated probes based on the previously defined event types and scopes.

REMINDS's *Monitoring Information Collection Layer* comprises probes and filters. Probes can be inserted statically in the monitored system, or dynamically (e.g., through dynamic Load-Time Weaving when using AspectJ [82] probes). Furthermore, to reduce the traffic overhead of sending events and event data, REMINDS allows to specify filters alongside with a probe. For example, a filter can be used to define a threshold for monitored values that are about to be sent to the monitoring infrastructure.

The *Distribution and Processing Layer* is to a large extent covered by the core layers of REMINDS. An Event Broker serves as a routing entity; it is responsible for receiving events from the diverse probes in the system and distributing them to subsequent processing, analysis, or visualization components. It uses a publish-subscribe pattern, meaning that arbitrary event consumers can subscribe to one or more sources and then process events, check constraints, or visualize system behavior for the user. Processors allow to (pre-)process received data before it is fed into the consistency checker (*Monitoring Information Transformation*). This includes, for example, aggregating events from one or more sources, calculating averages, minima, or maxima, or performing user-defined calculations on the data attached to the events.

REMINDS employs an incremental consistency checker [81] (ICC) to (on the fly) instantiate and evaluate the constraints based on the information received from the probes. The constraint checker uses the constraint definitions, which are translated to the underlying language (for REMINDS we translate those to Java code). When events are received at runtime, the ICC determines which constraint definitions are associated with the respective events and executes the corresponding Java code. As a result, the consistency checker generates a validation result. In case of a constraint violation, it provides detailed information on the events

involved in the violation and the data that caused the violation, which can be used for further diagnosis.

REMINDS furthermore provides a separate *Persistence* layer allowing to connect arbitrary storage solutions (prototypes include various SQL and NoSQL databases) to store events and event data. The persistence component subscribes to the event broker and receives events that need to be recorded. The component is also capable of publishing (previously stored) events to the event broker, e.g., for post-mortem analysis.

From the end-user perspective, engineers and service personnel are supported by various *Applications*. For instance, the REMINDS Monitoring Center [83] provides capabilities for managing constraints, scopes, and processors, and event and violation visualization. This includes visualizing high-level system behavior or event-flows between components. A dedicated view for the ICC (Evaluation Results View) allows engineers to analyze constraint violations and provides detailed information on the internal state of specific components.

Summary

Instantiating the reference architecture for REMINDS showed its suitability for describing this requirements monitoring approach. Again, certain variability can be observed (cf. Section 6.4): for example, the *Monitoring Definition* is split into two different parts (scopes and constraint definitions) to address the specifics of systems of systems.

7.3. ECoWare

The ECoWARE monitoring approach is specifically tailored towards service-based systems, yet its design makes it also suitable for other kinds of systems. It has been successfully deployed both for traditional Web Applications [84], and in the context of Smart Spaces [85]. The instantiated reference architecture for ECoWARE can be seen in Fig. 6.

To deploy an instance of the ECoWARE framework, the monitoring engineer must first define the rules that she wants ECoWARE to check at

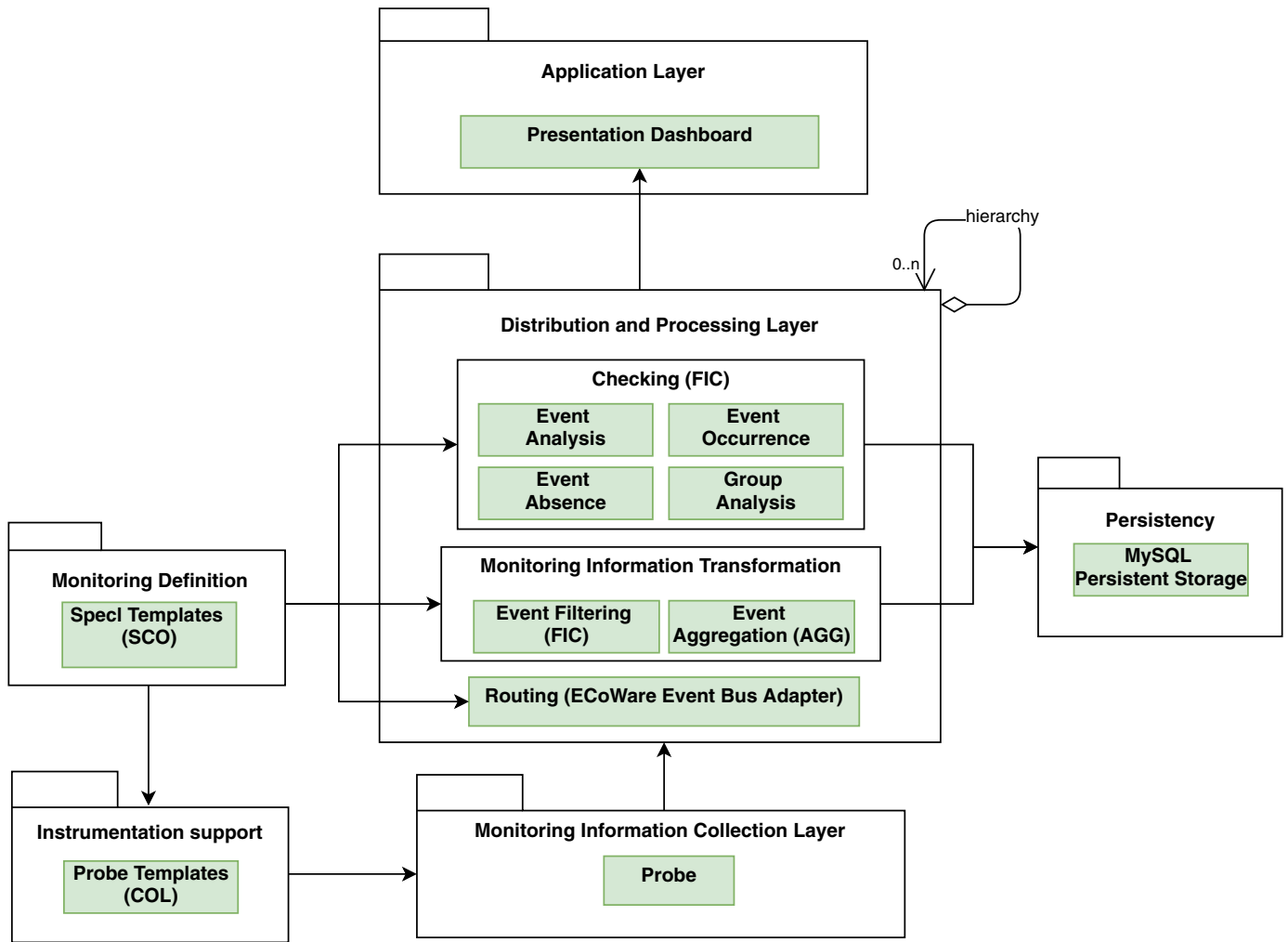


Fig. 6. Instantiated reference architecture for ECoWARE.

runtime. This is achieved using *Specil*, a special-purpose DSL for defining constraints over Service Data Objects (SDOs). *Specil Templates* ease this *Monitoring Definition* process.

The ECoWARE framework assumes that appropriate probes (*Monitoring Information Collection Layer*) are deployed to the monitored system to collect the low-level runtime data needed by its *Specil* directives, and that these probes output Service Data Objects (SDOs). Often times these probes will be application-specific. However, ECoWARE does provide some technology-specific *Probe Templates* as *Instrumentation Support* for probing service-based applications that employ Service Component Architecture (SCA), and Web-based applications that employ Servlets. The *Specil* constraints in the *Monitoring Definition* reference the SDOs output by the probes.

Specil definitions are used to inform the activities provided in ECoWARE's *Distribution and Processing Layer*. An instance of the ECoWARE framework will typically involve multiple *Distribution and Processing Layer* nodes, all interconnected by the ECoWARE Event Bus, thus exploiting the *hierarchy* relation shown in Fig. 6. Communication between the different nodes will depend on what the monitoring engineer specified in the *Specil* directives. While each node will have exactly one ECoWARE *Event Bus Adapter*, allowing it to send and receive SDO-based events to and from the bus, different nodes may have different combinations of *Monitoring Information Transformation* and *Checking* components (all these have 0: *n* cardinality).

ECoWARE provides *Monitoring Information Transformation* capabilities through its *Event Filtering* and *Event Aggregation* components. Event filtering allows us to verify whether a certain SDO has a data payload that satisfies a given constraint. If it does, it is sent back to the bus for further consumption by other *Checking* or *Monitoring Information Transformation* components; if it doesn't, it is discarded. Event Aggregation, on the other hand, is used to identify events that are relevant to one another, and to aggregate their payloads to create new higher-level events. These created events are also sent back to the bus for further consumption.

ECoWARE provides *Checking* capabilities via its *Event Analysis*, *Event Occurrence*, *Event Absence*, and *Group Analysis* components. Event analysis consists in checking the data payload of a given event to see whether it satisfies a given condition. The condition expresses something that needs to be true. Therefore, if the condition is satisfied the event is discarded; it is only sent back to the bus to notify the framework that an erroneous situation has occurred. Event Occurrence and Event Absence exploit complex event processing techniques to check whether certain event patterns occur, or not, within a given past/future time frame. Group Analysis is used to predicate over the combined behavior of a group of probes, e.g., “exactly/at least/at most *x* events, from different sources, satisfy a property *p* in a time frame *t*”.

Finally, ECoWARE provides optional *Persistency* capabilities based on MySQL for storing the events that are processed by the *Checking* and *Monitoring Information Transformation* components, and a *Presentation*

Dashboard on the Application Layer. This dashboard can be used to perform live charting, as well as online and offline violation drill-down analysis.

Summary

The main challenge when modeling ECOWARE's architecture was that it chains together multiple monitoring information filtering and transformation components (possibly deployed on different nodes) to progressively aggregate and manipulate the monitoring information to produce higher-level knowledge of a system's behavior. Nevertheless, thanks to the high degree of flexibility provided by our reference architecture, we were still able to concisely model ECOWARE's architecture. In particular, by using in combination the *hierarchy* relationship and the reference architecture's routing component—instantiated by the *ECOware Event Bus Adapter* component that is deployed to every *Distribution and Processing Layer* node in the monitoring system—events can be processed by any node in the monitoring system and forwarded for further consumption and manipulation.

7.4. Kieker

Kieker [10,25] is an extensible monitoring framework that has been applied to a rather diverse set of system types, including Java, Perl or Cobol programs. The Kieker core at the heart of the framework provides integrated probes, e.g., for performance monitoring or control-flow tracing, monitoring controllers and interfaces for realizing (external) extensions. Kieker uses a *Monitoring Definition* to instrument the target system, collects monitoring information in a *Monitoring Information Collection Layer* consisting of several central controller classes and, frequently, employs a *Persistency* mechanism to store the collected information for (offline) analysis. Further tasks, such as the visualization of the collected information, are realized by various plugins, e.g., for extracting or visualizing architectural models.

In contrast to the previous monitoring approaches, none of the authors of this paper was involved in the development of the Kieker approach or has detailed inside knowledge. Two of the authors took the opportunity of performing some shared discussion sessions with several Kieker developers and other people having detailed insight into the Kieker monitoring system.¹⁴ Rather than primarily detailing the Kieker monitoring framework, this section provides a summary of our discussions, in particular the experiences that we made when the Kieker experts instantiated our reference architecture for Kieker. The participants found the reference architecture and the exercise very helpful, in particular, as it helped them to arrive at a more homogeneous view of the current Kieker system and its capabilities.

First, the context of the work and the proposed reference architecture was described to the participants. Then each individual component and each connection from the reference architecture was discussed and compared to the realization of Kieker, more precisely the Kieker version 1.14 release, which in its architecture still complies to the technical description in [25]. In addition the question was raised repeatedly whether important aspects of the system were missing, i.e., not represented in the reference architecture. After the workshop, the participants reviewed our summary and provided clarifying feedback.

7.4.1. Findings

The first important observation was that Kieker is less a specific monitoring approach, but rather a generic framework to cover a wide range of different types of applications. Thus, it turned out to be quite challenging to discuss the actual specific capabilities of Kieker. In particular, Kieker is extensible via plug-ins that can—in principle—introduce arbitrary additional functionality. We thus focused on the Kieker capabilities

that are part of the core and main plugins that are commonly used and are part of the trunk of the Kieker repository.

The second observation was that the current Kieker implementation has a central component, called the *MonitoringController*, which plays an important role in many other functionalities. However, the proposed reference architecture does not have such a component. Here, the basic architectural approach of Kieker significantly differs from the reference architecture as it is apparently less organized as a data-flow architecture rather than as a call-return architecture with a main controller component. As it is not the goal of the reference architecture to represent the particular architectural styles of the representative monitoring systems, we refrained from introducing this into the reference architecture (or its Kieker instance shown in Fig. 7). Similarly, other components were not introduced, even though they were repeatedly mentioned (*AbstractMonitoringProbe*; pipes & filters framework). Another issue that was problematic was that sometimes the same terms are used in the Kieker definition, however, with a different meaning than in the reference architecture (e.g., monitoring definition). Based on the information we collected during the discussions of the various reference architecture elements, we developed a reference architecture instance of Kieker, which is shown in Fig. 7. This represents the Kieker capabilities, but does not represent the specific technical architecture in detail. The latter would be an inadequate expectation for a reference architecture. It should also be mentioned that the architecture describes Kieker as the very broad framework that it is nowadays.

As a result of the discussions we found that nearly all capabilities of Kieker could be well represented by the reference architecture. The only exception we found was that there should be an explicit link from the application layer to the monitoring definition, i.e., the application is capable of modifying the monitoring definition. Since this was not inconsistent with the previous understanding, we introduced this both into the reference architecture and the instance.

The instantiated architecture shows that Kieker covers most of the elements of the reference architecture, as it is itself a rather generic plugin framework and hence most of the capabilities can be realized. Noteworthy omissions are aggregation on the collection layer, the non-hierarchical distribution and processing layer and the omission of various backlinks. However, due to the generality of the framework some plugins could actually introduce these capabilities on their own.

Summary

The main challenges in modeling Kieker as an instance of the reference architecture were (i) to determine which possible capabilities of the plugins should actually be considered part of the architecture and (ii) to map the different terminology between both models.

As part of the discussion it became clear that the reference architecture was a very useful blueprint for discussing capabilities of Kieker. It also allowed to identify some elements which Kieker currently does not provide, which might be a basis for future development.

Summary and threats to validity

We successfully demonstrated that our reference architecture is useful to describe the key components and layers of a software monitoring approach from an architectural perspective. While our reference architecture also could be used to compare different monitoring approaches, in this paper we focused on using the domain model to compare (47) approaches. Using the reference architecture to compare approaches would, however, be comparably easy and mainly mean to put the instances created for particular monitoring approaches side by side to then check which elements of the reference architecture are part of which instance and in which form. The challenging part is creating the actual instances. This requires detailed knowledge about approaches' architectures, information that is typically not available in associated publications and other available sources.

Table 9 provides a summary and overview of how each element of the reference architecture was instantiated for the analyzed approaches

¹⁴ The participants of the discussions were: André van Hoorn, Reiner Jung, Dušan Okanović, Sören Henning, Thomas F. Düllmann, and Holger Knoche.

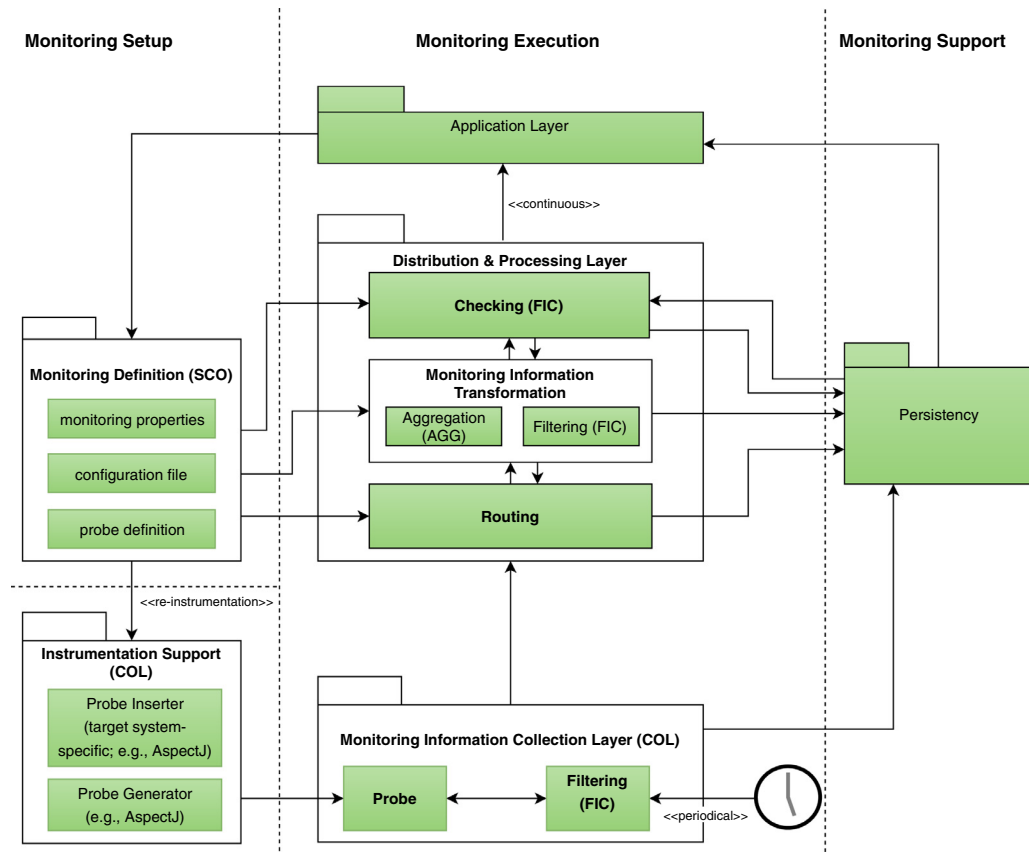


Fig. 7. Instantiated reference architecture for Kieker.

Table 9
Summary of instantiation information for examples.

Ref. Arch.	SPASS-meter	QualiMaster	REMINDS	ECOWARE	Kieker
Monitoring Setup					
• Mon. Def.	Mon. Scope	Mon. Scope, Constr. Model	Mon. Scope, Constr. Templ., Constr. Defs.	Spec. Templ.	Monitoring Props; Probe Def. (target-spec.)
• Instr. Supp.					
* Probe Ins.	Probe Inse.	Probe Ins.	AspectJ Gen.	—	Probe Def.
* Probe Gen.	Probe Gen.	Probe Gen.	Probe Templ.	Probe Templ.	specific DSL
Monitoring Execution					
• Application	Presentation Plugins	Adaptation (QM, EASy)	Mon. Center, Eval. Res. View	Presentation Dashboard	via plugins or externally
• Distrib. & Proc.					
* Checking	—	Constraint Checking	Incremental Consistency Checker	Event Analysis, Occurrence & Absence, Group Analysis	Pipes&Filters Framework + External Components
* Mon. Inf. Tra.					
· Aggregation	(Online) Aggregation	Aggregation (QM)	Processors	Event Aggregation	Pipes&Filters Framework
· Filtering	—	—	Processors	Event Filtering	Pipes&Filters Framework
* Routing	(Internal) Routing	Routing (SPASS-meter), Routing (QM event bus)	Routing (event broker)	Routing (ECOWare Event Bus Adapter)	Reader, Writer Framework
• Mon. Inf. Coll.					
* Probe	Probe	Probe	Probe	Probe	Abstract Monitoring Probe and Instances in Probes (rare)
* Filtering	—	—	in Probes	—	—
* Aggregation	Aggregation	Aggregation	—	—	—
Monitoring Support					
• Persistency	—	Monitoring Log, Adapt. Log	DB-Connector (SQL, NoSQL)	MySQL Persistent Storage	files; several database systems
Essential Differences					
	re-instrumentation at runtime	re-instrumentation at runtime	takes persisted information into account	distributed hierarchy of processing layer	application may modify monitoring definition

(and which elements are not supported) to demonstrate commonalities and differences.

Regarding *threats to validity*, similar to the domain model, the development of our reference architecture was to a large extent influenced by the 47 approaches we analyzed, specifically the information reported in the respective scientific publications. However, as described above, scientific publications often do not provide detailed (enough) information about the capabilities of approaches, especially with regard to their architectures. Instantiating our reference architecture thus requires the involvement of the original developers of the approaches, like we did for all five instances we created.

A specific threat to the validity of our reference architecture thus is that we mainly instantiated it for our own approaches. While this allows to demonstrate its basic applicability, it must be considered a threat. To mitigate this threat, we did also instantiate our reference architecture together with Kieker developers on their approach. However, as we moderated this session, we might have influenced this as well. Instantiating it for other approaches might reveal missing components or required changes in the future. For example, a very small change (link between application and monitoring definition) was actually required when doing it for Kieker, and this could also be the case for future instances.

Applying our reference architecture to other approaches will require detailed information about the architectures of these approaches and/or discussions with the developers, similar to what we did for Kieker. We think this article will be a good basis for such developers to create “their own” instance and compare it with the other instances we created and describe in this article.

8. Related work

Our work aims to foster the integration of resource monitoring and requirements monitoring by presenting both a domain model and a reference architecture. Therefore, in this section we discuss existing research on domain analysis, as well as examples of domain models developed for specific fields of software engineering. We also discuss other examples of reference architectures in different domains. Finally, we briefly compare our work to existing surveys in the area of software monitoring.

8.1. Domain analysis in software engineering

Numerous techniques have been proposed to support domain analysis in software engineering. For example, the Draco approach [86] proposes a language to define objects and operations of programs in a specific domain. The resulting domain model can then be reused for new programs to be developed. However, the purpose of this method is to organize reusable software components at the technical level by problem area or domain, which differs significantly from our research context.

Domain analysis also plays a central role in the context of software product lines. For instance, Khurum and Gorschek [87] provide a comprehensive systematic review of domain analysis solutions for product lines. Schmid [55] analyzes the existing body of knowledge on product line scoping. A widely used approach is the Feature-Oriented Domain Analysis (FODA) [54]. The focus of the method is to identify “prominent or distinctive features of software systems” in a domain, which lead to the creation of a set of software products defining the domain. Again, the main focus of our work was not to analyze the commonalities and variability of software features in a domain, but to systematically analyze an important field of research.

A similar goal was pursued by authors in different domains of software engineering, following a research method more comparable to the approach taken in our paper. For instance, Buckley et al. [88] propose a taxonomy of software change intended as a framework to structure methods and tools within the domain of software evolution. Czarnecki et al. [89] attempt to clarify the relation between feature modeling and

decision modeling in the research field of variability modeling as a basis for exploring potential synergies. Kanstren [90] presents a taxonomy of runtime invariance in software behavior. While the specific details of the research methods and the used terminology in these papers differ, the authors generally follow a similar approach to develop, refine, and check a domain model by identifying, structuring, and cross-checking concepts of their respective domains.

Our research method is also related to the Thematic Synthesis approach by Cruzes and Dyba [91]. The authors extended existing methods for Thematic Analysis. Their five-step approach aims at identifying recurring themes in primary studies, analyzing these themes, and drawing conclusions in systematic reviews. Specifically, the approach aims at (i) extracting data from primary studies, (ii) coding data by identifying interesting concepts in a systematic manner, (iii) translating codes into themes to organize the codes, (iv) creating a model of higher-order themes, and (v) assessing the trustworthiness of the results. While these steps map to the steps of our research method at the surface, we did not use a coding process as in the Thematic Synthesis. Also, in our case the literature analysis was preceded by a brainstorming activity to create an initial domain model, which was later refined based on the literature analysis.

8.2. Reference architectures

A number of approaches exist to inform the development of reference architectures, similar to the approach we pursued in our paper. For instance, Nakagawa et al. [92] propose the RAModel (Reference Architecture Model), that defines a comprehensive guideline to define reference architectures, as well as their key components and relationships. The description of our reference architecture emphasizes in particular software elements in the Infrastructure group of the RAModel, while our domain model addresses the scope and requirements in the Application group of the RAModel. Galster and Avgeriou [93] present a method for systematically designing empirically-grounded reference architectures. Our research method tries to cover the key steps proposed in this method to ensure the empirical foundation—e.g., our literature analysis and data collection of existing monitoring systems—as well as the empirical validity by demonstrating the applicability of our reference architecture to several existing monitoring solutions. Angelov et al. [94] investigate a number of reference architectures and identify the level of congruence between their goals, context, and design as the main reason for success. Following their approach, we regard our reference architectures as a Type 5. Such architectures are designed to facilitate the design of architectures of systems that will become needed in the future. While our reference architectures defines the components required in a system implementing it, we do currently not describe the algorithms supporting the operation of the components, and protocols demonstrating the interactions among the components. Authors have also developed reference architectures for specific domains. For instance, Batory et al. present ADAGE, a component-based reference architecture for avionics software synthesis [95]. We focus on the domain of software monitoring.

8.3. Surveys and reviews on monitoring

The heterogeneity of existing software monitoring approaches already led to numerous research papers which tried to structure and analyze the domain. For instance, Delgado et al. [21] present a taxonomy intended for understanding developments in runtime software fault-monitoring. The taxonomy comprises elements essential for building a monitoring system, i.e., a property specification language, a monitoring mechanism, as well as components for capturing and communicating monitoring results. The elements of the taxonomy are also covered by our domain model, however, our domain model is more comprehensive regarding the resource monitoring aspects. There is also some overlap regarding the assessed approaches and tools, however, our analysis covers more and especially also more recent approaches published

since 2004. In the area of runtime verification Dwyer et al. [20] aim to support users who need to specify the requirements in specification languages of verification tools. The authors use a pattern-based approach to present, codify and reuse property specifications for runtime verification. Specifically, the paper presents a pattern hierarchy based on a collection of property specification. Following this idea Autili et al. [22] present a framework combining qualitative, real-time, and probabilistic property specification patterns. They report a unified catalogue combining already known and 40 newly identified or extended patterns. Our domain model element Expressiveness and its subelements cover different aspects of properties that are supported in current specification languages. However, we do not provide concrete patterns as in these publications. The survey of Rabiser et al. [4] provides a comparison framework for runtime monitoring approaches that is based on a literature analysis and existing taxonomies for monitoring languages and patterns. The domain model presented in this paper is far more comprehensive, particularly as it also covers resource monitoring aspects. Other surveys investigate software monitoring in specific domains. Vierhauser et al. [5] investigate requirements monitoring in the context of systems-of-systems architectures. Specifically, the paper discusses requirements monitoring frameworks identified in a systematic literature review, organized by the four layers of Robinson [9]. Some of the criteria used in this paper can also be found in our domain model, but the focus of paper is on assessing to what extent existing approaches support challenges of monitoring in system of systems. Aceto et al. [51] provide a survey for the field of cloud monitoring. They discuss the motivations, basic concepts, key properties, open issues, and future directions. The survey by Hu et al. [52] emphasizes resource monitoring and prediction strategies for the field of grid computing. Both of these surveys focus on resource monitoring, while we also focus on requirements monitoring with our domain model. In conclusion, while there are several surveys of requirements or resource monitoring approaches, none of these existing works focused on providing a comprehensive domain model for requirements monitoring and resource monitoring (and potentially other monitoring approaches in the future).

9. Conclusions and future work

We presented a domain model and a reference architecture for software monitoring. The model and architecture are based on (i) our analysis of monitoring approaches from the fields of requirements monitoring and resource monitoring, (ii) earlier experiences developing a systematic literature review and a survey of these domains, and (iii) our previous work on developing monitoring approaches for different purposes. The resulting domain model defines the key elements of a software monitoring approach and provides a common terminology to discuss approaches and tools coming from different communities. We assessed the coverage of the domain model by analyzing 47 monitoring approaches, i.e., 27 requirements monitoring approaches and 20 resource monitoring approaches. We also developed a reference architecture for monitoring systems. We evaluated this reference architecture by instantiating it for four of our own monitoring approaches. We additionally evaluated it for a fifth approach, in a session with experienced developers of the Kieker tool.

Our efforts structure the field of software monitoring and bridge the gap (regarding terminology and key elements) between two of its main communities, i.e., requirements monitoring and resource monitoring. Overall we can conclude that these two communities have more commonalities than differences and that a better integration of the two fields is promising. We are confident that our model and architecture could also be useful when discussing monitoring approaches from other communities, although adaptations to the model and architecture might be necessary.

Both researchers and practitioners can benefit from our work, i.e., researchers can use the domain model to understand the domain of

software monitoring and categorize and motivate their own research. They can, for instance, check our analysis of 47 approaches to identify gaps and commonalities and differences between resource and requirements monitoring approaches. They can also use the domain model to analyze their own monitoring approach. Based on the description of the layers and components our reference architecture provides and using the five instances we already created as examples, both researchers and practitioners are supported in developing new monitoring solutions. Domain model and reference architecture also support and motivate developing integrated solutions, e.g., monitoring approaches that support both requirements and resource monitoring.

In our future work we aim to investigate other monitoring communities, such as runtime verification and network monitoring. We also plan to ask experts in the different fields for their opinion on our domain model and reference architecture. Using our online spreadsheet [26] other researchers can easily check the information we extracted and suggest changes as well as additions. We also plan to ask practitioners—e.g., developers of application performance monitoring tools—for their input to help us make our model and architecture more relevant for their work.

Acknowledgments

The financial support by the [Austrian Federal Ministry for Digital and Economic Affairs](#), the National Foundation for Research, Technology and Development, and Primetals Technologies (via the [Christian Doppler Lab MEVSS](#)) is gratefully acknowledged (Rabiser). Michael Vierhauser's work was funded by the [Austrian Science Fund \(FWF\)](#) under Grant No. [J3998-N31](#). This work was also partially supported by the QualiMaster (grant [619525](#)) funded by the [European Commission](#) in the 7th framework programme. Any opinions expressed herein are solely by the authors and not of the EU.

We would also like to thank the Kieker experts, who participated in our evaluation discussions, namely André van Hoorn, Reiner Jung, Dušan Okanović, Sören Henning, Thomas F. Düllmann, and Holger Knoche.

References

- [1] E.A. Lee, Cyber physical systems: design challenges, in: 11th IEEE Int'l Symp. on Object-oriented Real-time Distributed Computing, IEEE, 2008, pp. 363–369.
- [2] M. Armbrust, A. Fox, R. Griffith, A.D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, M. Zaharia, A view of cloud computing, *Commun. ACM* 53 (4) (2010) 50–58.
- [3] C.B. Nielsen, P.G. Larsen, J. Fitzgerald, J. Woodcock, J. Peleska, Systems of systems engineering: basic concepts, model-based techniques, and research directions, *ACM Comput. Surv.* 48 (2) (2015) 18:1–18:41.
- [4] R. Rabiser, S. Guinea, M. Vierhauser, L. Baresi, P. Grünbacher, A comparison framework for runtime monitoring approaches, *J. Syst. Softw.* 125 (2017) 309–321.
- [5] M. Vierhauser, R. Rabiser, P. Grünbacher, Requirements monitoring frameworks: a systematic review, *Inf. Softw. Technol.* 80 (2016) 89–109.
- [6] H. Eichelberger, C. Qin, K. Schmid, C. Niederée, Adaptive application performance management for big data stream processing, *Softwaretechnik* 35 (3) (2015) 35–37.
- [7] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Softw.* 33 (3) (2016) 35–41.
- [8] R. Calinescu, C. Ghezzi, M. Kwiatkowska, R. Mirandola, Self-adaptive software needs quantitative verification at runtime, *Commun. ACM* 55 (9) (2012) 69–77.
- [9] W.N. Robinson, A requirements monitoring framework for enterprise systems, *Require. Eng.* 11 (1) (2006) 17–41.
- [10] A. van Hoorn, J. Waller, W. Hasselbring, Kieker: a framework for application performance monitoring and dynamic software analysis, in: 3rd joint ACM/SPEC Int'l Conf. on Performance Engineering, ACM, 2012, pp. 247–248.
- [11] H. Eichelberger, K. Schmid, Flexible resource monitoring of Java programs, *J. Syst. Softw.* 93 (2014) 163–186.
- [12] P. Lengauer, V. Bitto, H. Mössenböck, Accurate and efficient object tracing for java applications, in: 6th ACM/SPEC Int'l Conf. on Performance Engineering, ACM, 2015, pp. 51–62.
- [13] H. Muccini, A. Polini, F. Ricci, A. Bertolino, Monitoring architectural properties in dynamic component-based systems, in: 10th Int'l Symp. on Component-Based Software Engineering, Springer, 2007, pp. 124–139.
- [14] M. Völz, B. Koldehofe, K. Rothermel, Supporting strong reliability for distributed complex event processing systems, in: 13th Int'l Conf. on High Performance Computing and Communication, 2011, pp. 477–486.

- [15] C. Ghezzi, A. Mocci, M. Sangiorgio, Runtime monitoring of component changes with Spy@Runtime, in: 34th Int'l Conf. on Software Engineering, 2012, pp. 1403–1406.
- [16] K. Havelund, G. Roşu, Monitoring Java programs with Java Pathexplorer, *Electron. Notes Theor. Comput. Sci.* 55 (2) (2001) 200–217.
- [17] S. Guinea, G. Kecs-kemeti, A. Marconi, B. Wetzstein, Multi-layered monitoring and adaptation, in: 9th Int'l Conf. on Service-Oriented Computing, 2011, pp. 359–373.
- [18] M. Vierhauser, R. Rabiser, P. Grünbacher, K. Seyerlehner, S. Wallner, H. Zeisel, Re-Minds: A flexible runtime monitoring framework for systems of systems, *J. Syst. Softw.* 112 (2016) 123–136.
- [19] M. Mansouri-Samani, M. Sloman, Monitoring distributed systems, *IEEE Netw.* 7 (6) (1993) 20–30.
- [20] M. Dwyer, G. Avrunin, J. Corbett, Patterns in property specifications for finite-state verification, in: Int'l Conf. on Software Engineering, 1999, pp. 411–420.
- [21] N. Delgado, A.Q. Gates, S. Roach, A taxonomy and catalog of runtime software-fault monitoring tools, *IEEE Trans. Softw. Eng.* 30 (12) (2004) 859–872.
- [22] M. Autili, L. Grunske, M. Lumpe, P. Pelliccione, A. Tang, Aligning qualitative, real-time, and probabilistic property specification patterns using a structured english grammar, *IEEE Trans. Softw. Eng.* 41 (7) (2015) 620–638.
- [23] L. Baresi, S. Guinea, Event-based multi-level service monitoring, in: 20th Int'l Conf. on Web Services, IEEE, 2013, pp. 83–90.
- [24] H. Eichelberger, C. Qin, K. Schmid, From resource monitoring to requirements-based adaptation: an integrated approach, in: 8th ACM/SPEC Int'l Conf. on Performance Engineering Companion, ACM, 2017, pp. 91–96.
- [25] A. van Hoorn, M. Rohr, W. Hasselbring, J. Waller, J. Ehlers, S. Frey, D. Kieselhorst, Continuous Monitoring of Software Services: Design and Application of the Kieker Framework, Technical Report, Kiel University, 2009.
- [26] Spreadsheet of our Domain Model with Information extracted from (47) Monitoring Approaches (2019), (https://docs.google.com/spreadsheets/d/1DrhwEF_BKexAKiBBlwA4YFG_xG0y9utYvKHdNEjxCa/). [Online; accessed 30-Jan-2019; version from this date on <https://dx.doi.org/10.5281/zenodo.1964937>].
- [27] S. Fickas, M.S. Feather, Requirements monitoring in dynamic environments, in: 2nd IEEE Int'l Symp. on Requirements Engineering, 1995, pp. 140–147.
- [28] Y. Wang, S.A. McIlraith, Y. Yu, J. Mylopoulos, Monitoring and diagnosing software requirements, *Autom. Softw. Eng.* 16 (1) (2009) 3–35.
- [29] B. Li, S. Ji, L. Liao, D. Qiu, M. Sun, Monitoring web services for conformance, in: 7th Int'l Symp. on Service Oriented System Engineering, IEEE, 2013, pp. 92–102.
- [30] A. Dragomir, H. Lichter, Run-time monitoring and real-time visualization of software architectures, in: 20th Asia-Pacific Software Engineering Conf., vol. 1, IEEE, 2013, pp. 396–403.
- [31] J. Simmonds, S. Ben-David, M. Chechik, Monitoring and recovery for web service applications, *Computing* 95 (3) (2013) 223–267.
- [32] E. Mulo, U. Zdun, S. Dustdar, An event view model and DSL for engineering an event-based SOA monitoring infrastructure, in: 4th ACM Int'l Conf. on Distributed Event-Based Systems, ACM, 2010, pp. 62–72.
- [33] A. Contreras, K. Mahub, MORPED: monitor rules for proactive error detection based on run-time and historical data, in: 5th Int'l Conf. on the Applications of Digital Information and Web Technologies, 2014, pp. 28–35.
- [34] A.B. Hamida, A. Bertolino, A. Calabrò, G. De Angelis, N. Lago, J. Lesbegueres, Monitoring service choreographies from multiple sources, in: 4th Int'l WS on Software Engineering for Resilient Systems, Springer, 2012, pp. 134–149.
- [35] F. Chen, M. d'Amorim, G. Roşu, Checking and correcting behaviors of Java programs at runtime with Java-MOP, *Electron. Notes Theor. Comput. Sci.* 144 (4) (2006) 3–20.
- [36] P. Doherty, J. Kvarnström, F. Heintz, A temporal logic-based planning and execution monitoring framework for unmanned aircraft systems, *Auton. Agent. Multi Agent Syst.* 19 (3) (2009) 332–377.
- [37] J. Shao, H. Wei, Q. Wang, H. Mei, A runtime model based monitoring approach for cloud, in: 3rd Int'l Conf. on Cloud Computing, IEEE, 2010, pp. 313–320.
- [38] A. Chawla, A. Orso, A generic instrumentation framework for collecting dynamic information, *SIGSOFT Softw. Eng. Notes* 29 (5) (2004) 1–4.
- [39] J. Ehlers, W. Hasselbring, A self-adaptive monitoring framework for component-based software systems, in: I. Crnkovic, V. Gruhn, M. Book (Eds.), 5th European Conf. on Software Architecture, Springer Berlin Heidelberg, 2011, pp. 278–286.
- [40] P. Bellavista, A. Corradi, C. Stefanelli, Java for on-line distributed monitoring of heterogeneous systems and services, *Comput. J.* 45 (6) (2002) 595–607.
- [41] W. Binder, J. Hulaas, A portable CPU-management framework for Java, *IEEE Internet Comput.* 8 (5) (2004) 74–83.
- [42] W. Binder, J. Hulaas, Extending standard Java runtime systems for resource management, in: T. Gschwind, C. Mascolo (Eds.), 4th Int'l WS on Software Engineering and Middleware, Springer Berlin Heidelberg, 2005, pp. 154–169.
- [43] G. Czajkowski, T. von Eicken, JRes: a resource accounting interface for Java, in: 13th ACM SIGPLAN Conf. on Object-oriented Programming, Systems, Languages, and Applications, ACM, 1998, pp. 21–35.
- [44] D. Okanović, A. van Hoorn, C. Heger, A. Wert, S. Siegl, Towards performance tooling interoperability: an open format for representing execution traces, in: 13th European WS on Computer Performance Engineering, Springer International Publishing, 2016, pp. 94–108.
- [45] W. Zhou, O. Sokolsky, B.T. Loo, I. Lee, DMac: distributed monitoring and checking, *RV* 5779 (2009) 184–201.
- [46] D. Breitgand, D. Dolev, D. Raz, G. Shaviner, Facilitating efficient and reliable monitoring through HAMSA, in: Integrated Network Management VIII, Springer, 2003, pp. 263–276.
- [47] K.G. Anagnostakis, M. Greenwald, S. Ioannidis, S. Miltchev, Open packet monitoring on FLAME: safety, performance, and applications, in: IFIP Int'l Working Conf. on Active Networks, Springer, 2002, pp. 120–131.
- [48] H. Gunadi, A. Tiu, Efficient runtime monitoring with metric temporal logic: a case study in the android operating system, in: 19th Int'l Symp. on Formal Methods, Springer International Publishing, 2014, pp. 296–311.
- [49] B. Ouchenne, O. Koné, Specifying and analysing run-time security policies for time dependant services, in: First Int'l WS on Security and Privacy Preserving in e-Societies, ACM, 2011, pp. 26–33.
- [50] A.P. Sistla, M. Žefran, Y. Feng, Runtime monitoring of stochastic cyber-physical systems with hybrid state, in: Int'l Conf. on Runtime Verification, Springer, 2011, pp. 276–293.
- [51] G. Aceto, A. Botta, W. de Donato, A. Pescapé, Cloud monitoring: a survey, *Comput. Netw.* 57 (9) (2013) 2093–2115.
- [52] L. Hu, X. Cheng, X. Che, Survey of grid resource monitoring and prediction strategies, *IJIIP* 1 (2) (2010) 78–85.
- [53] W.C. Lim, Managing Software Reuse, Prentice Hall PTR, 1998.
- [54] K. Kang, S. Cohen, J. Hess, W. Nowak, S. Peterson, Feature-oriented Domain Analysis (FODA) Feasibility Study, Technical Report, CMU/SEI-90TR-21, 1990.
- [55] K. Schmid, Scoping software product lines—an analysis of an emerging technology, in: 1st Software Product Line Conf., Kluwer, 2000, pp. 513–532.
- [56] C. Ruz, F. Baude, B. Sauvan, Component-based generic approach for reconfigurable management of component-based SOA applications, in: 3rd Int'l WS on Monitoring, Adaptation and Beyond, ACM, 2010, pp. 25–32.
- [57] M. Comuzzi, G. Spanoudakis, A framework for hierarchical and recursive monitoring of service based systems, in: 4th Int'l Conf. on Internet and Web Applications and Services, IEEE, 2009, pp. 383–388.
- [58] C. Zeginis, K. Kritikos, P. Garefalakis, K. Konsolaki, K. Magoutis, D. Plexousakis, Towards cross-layer monitoring of multi-cloud service-based applications, in: 2nd European Conf. on Service-Oriented and Cloud Computing, Springer, 2013, pp. 188–195.
- [59] F. Chen, G. Roşu, MOP: an efficient and generic runtime verification framework, in: 22nd Annual ACM SIGPLAN Conf. on Object-oriented Programming Systems and Applications, ACM, 2007, pp. 569–588.
- [60] K. Chan, I. Poernomo, H. Schmidt, J. Jayaputera, A model-oriented framework for runtime monitoring of nonfunctional properties, in: 1st Int'l Conf. on the Quality of Software Architectures, Springer, 2005, pp. 38–52.
- [61] A. Michlmayr, F. Rosenberg, P. Leitner, S. Dustdar, Comprehensive QoS monitoring of web services and event-based SLA violation detection, in: 4th Int'l WS on Middleware for Service Oriented Computing, ACM, 2009, pp. 1–6.
- [62] D. Źmuda, M. Psiuk, K. Zieliński, Dynamic monitoring framework for the SOA execution environment, *Procedia Comput. Sci.* 1 (1) (2010) 125–133.
- [63] A.J. Ramirez, B.H. Cheng, P.K. McKinley, Adaptive monitoring of software requirements, in: 1st Int'l WS on Requirements@Run.Time, IEEE, 2010, pp. 41–50.
- [64] A.Q. Gates, S. Roach, O. Mondragon, N. Delgado, DynaMICs: comprehensive support for run-time monitoring, *Electron. Notes Theor. Comput. Sci.* 55 (2) (2001) 164–180.
- [65] P. Bellavista, A. Corradi, C. Stefanelli, Monitor and control of mobile agent applications, in: ACM OOPSLA WS on Experiences with Autonomous Mobile Objects and Agent Based Systems, 2000, pp. 1–8.
- [66] K. Bratanis, D. Dranidis, A.J. Simons, An extensible architecture for run-time monitoring of conversational web services, in: 3rd Int'l WS on Monitoring, Adaptation and Beyond, ACM, 2010, pp. 9–16.
- [67] V. Bitto, P. Lengauer, H. Mössenböck, Efficient rebuilding of large Java heaps from event traces, in: Principles and Practices of Programming on The Java Platform, ACM, 2015, pp. 76–89.
- [68] M. Shaw, D. Garlan, Software Architecture, vol. 101, Prentice Hall Englewood Cliffs, 1996.
- [69] J. Waller, W. Hasselbring, A comparison of the influence of different multi-core processors on the runtime overhead for application-level monitoring, in: Int'l Conf. on Multicore Software Engineering, Performance, and Tools, Springer Berlin Heidelberg, 2012, pp. 42–53.
- [70] M. Shaw, What makes good research in software engineering? *Int. J. Softw. Tools Technol. Trans.* 4 (1) (2002) 1–7.
- [71] T. Miettinen, D. Pakkala, M. Hongisto, A method for the resource monitoring of OSGi-based software components, in: 34th Euromicro Conf. Software Engineering and Advanced Applications, IEEE, 2008, pp. 100–107.
- [72] C. Becker, H. Kulovits, M. Kraxner, R. Gottardi, A. Rauber, An extensible monitoring framework for measuring and evaluating tool performance in a service-oriented architecture, in: 9th Int'l Conf. on Web Engineering, Springer, 2009, pp. 221–235.
- [73] F. Martinelli, P. Mori, T. Quillinan, C. Schaefer, A runtime monitoring environment for mobile Java, in: Int'l Conf. on Software Testing Verification and Validation WS, IEEE, 2008, pp. 270–278.
- [74] W. Binder, J-SEAL2—a secure high-performance mobile agent system, *Electron. Commerce Res.* 1 (1) (2001) 131–148.
- [75] R. Jung, C. Wulf, Advanced typing for the Kieker instrumentation languages, *Softwaretechnik* 36 (4) (2016) 1–3.
- [76] E.S. Al-Shaer, Programmable agents for active distributed monitoring, in: 10th IFIP/IEEE Int'l WS on Distributed Systems, Springer, 1999, pp. 19–32.
- [77] F. Fittkau, W. Hasselbring, Elastic application-level monitoring for large software landscapes in the cloud, in: 4th European Conf. on Service Oriented and Cloud Computing, Springer International Publishing, 2015, pp. 80–94.
- [78] H. Eichelberger, S. El-Sharkawy, C. Kröher, K. Schmid, EASY-producer: product line development for variant-rich ecosystems, in: 18th Int'l Software Product Line Conf., Volume 2, ACM, 2014, pp. 133–137.
- [79] H. Eichelberger, A. Sass, K. Schmid, From reproducibility problems to improvements: a journey, *Softwaretechnik* 36 (4) (2016) 43–45.
- [80] M. Vierhauser, R. Rabiser, P. Grünbacher, B. Aumayr, A requirements monitoring model for systems of systems, in: 23rd IEEE Int'l Requirements Engineering Conf., IEEE, 2015, pp. 96–105.

- [81] R. Rabiser, J. Thanhofer-Pilisch, M. Vierhauser, P. Grünbacher, A. Egyed, Developing and evolving a DSL-based approach for runtime monitoring of systems of systems, *Autom. Softw. Eng.* 25 (4) (2018) 875–915.
- [82] G. Kiczales, M. Mezini, Aspect-oriented programming and modular reasoning, in: 27th Int'l Conf. on Software Engineering, ACM, 2005, pp. 49–58.
- [83] L.M. Kritzinger, T. Krismayer, M. Vierhauser, R. Rabiser, P. Grünbacher, Visualization support for requirements monitoring in systems of systems, in: 32th IEEE/ACM Int'l Conf. on Automated Software Engineering, IEEE, 2017, pp. 889–894.
- [84] F. Seracini, M. Menarini, I. Krueger, L. Baresi, S. Guinea, G. Quattrocchi, A comprehensive resource management solution for web-based systems, in: 11th Int'l Conf. on Autonomic Computing, USENIX Association, 2014, pp. 233–239.
- [85] L. Baresi, S. Guinea, Event-based monitoring of service-oriented smart spaces (Invited Paper), in: 8th IEEE Int'l Conference on Service-Oriented Computing, IEEE, 2015, pp. 123–130.
- [86] J.M. Neighbors, The Draco approach to constructing software from reusable components, *IEEE TSE* 10 (5) (1984) 564–574.
- [87] M. Khurum, T. Gorschek, A systematic review of domain analysis solutions for product lines, *J. Syst. Softw.* 82 (12) (2009) 1982–2003.
- [88] J. Buckley, T. Mens, M. Zenger, A. Rashid, G. Kniesel, Towards a taxonomy of software change, *J. Softw. Mainten.* 17 (5) (2005) 309–332.
- [89] K. Czarnecki, P. Grünbacher, R. Rabiser, K. Schmid, A. Wasowski, Cool features and tough decisions: a comparison of variability modeling approaches, in: 6th Int'l WS on Variability Modelling of Software-Intensive Systems, 2012, pp. 173–182.
- [90] T. Kanstrén, A systematic review and taxonomy of runtime invariance in software behaviour, *Int. J. Adv. Softw.* 4 (3 and 4) (2011) 256–274.
- [91] D.S. Cruzes, T. Dybå, Recommended steps for thematic synthesis in software engineering, in: 5th Int'l Symp. on Empirical Software Engineering and Measurement, 2011, pp. 275–284.
- [92] E.Y. Nakagawa, F. Oquendo, M. Becker, Ramodel: a reference model for reference architectures, in: 2012 Joint Working IEEE/IFIP Conf. on Software Architecture and European Conference on Software Architecture, 2012, pp. 297–301.
- [93] M. Galster, P. Avgeriou, Empirically-grounded reference architectures: a proposal, in: Joint ACM SIGSOFT Conf. – QoSA and ACM SIGSOFT Symposium – ISARCS on Quality of Software Architectures – QoSA and Architecting Critical Systems, ACM, 2011, pp. 153–158.
- [94] S. Angelov, P.W.P.J. Grefen, D. Greefhorst, A classification of software reference architectures: analyzing their success and effectiveness, in: Joint Working IEEE/IFIP Conf. on Software Architecture, 2009, pp. 141–150.
- [95] D. Batory, Creating reference architectures: an example from avionics, in: ACM-SIGSOFT Symp. on Software Reusability, 1995, pp. 27–37.